ALS THERAPY DEVELOPMENT INSTITUTE

THE DRUG DISCOVERY ENGINE FOR ALS

OHDSI Training Day 6

Introduction to HADES

Danielle Boyce, MPH, DPA
Principal Investigator, RWE

Welcome

Gratitude

- Parsa Mirhaji
- Rimma Belenkaya
- Chenyre Okpara
- Jessica Sommer
- Ferris Hussein
- Alex Low
- Pavel Goriacko

- OHDSI Symposium Trainers for sharing lecture materials which were adapted for this training series

…and all of you!

THE DRUG DISCOVERY ENGINE FOR ALS

ALS THERAPY DEVELOPMENT INSTITUTE

Thank you so much for inviting me to speak today. Here are just a few of the people I want to thank for their help and encouragement.

Learning Objectives

- Build confidence using HADES
- Understand the relationship between HADES and ATLAS
- Identify the core HADES packages and their roles
- Understand how to run basic analyses using HADES packages

Time	Session
9:30 – 10:00	Kahoot! and Review
10:00 – 10:30	The OHDSI Mindset
10:30 – 11:30	Beyond ATLAS: HADES and the OHDSI Analytics Ecosystem
11:30 – 11:45	Break
11:45 – 12:30	Patient-level Prediction: Hands-on Demo
12:30 – 1:00	Concluding Thoughts and Staying Connected

Kahoot!

Before we go further, I want to get a sense of the room.
This Kahoot is **anonymous** and **not a test** — it's just to help me pace things and emphasize what will be most useful today.
There are no right or wrong answers. Please answer based on how *comfortable* you feel right now.
(We will explicitly turn on **"Hide leaderboard"** and **"No points"** in Kahoot.)

This is really helpful — thank you.
You can see there's a wide range of experience in the room, which is completely normal.
I'll focus on explaining *connections and concepts* clearly, and I'll flag where deeper technical knowledge becomes useful — but not required — as we go.

Housekeeping

- When I say “Atlas” I mean SEARCH
- When I say “cohort definition” I mean criteria definition
- When you read “OHDSI Methods Library” it means HADES (new name)
- https://ohdsi.github.io/CommonDataModel/cdm54.html

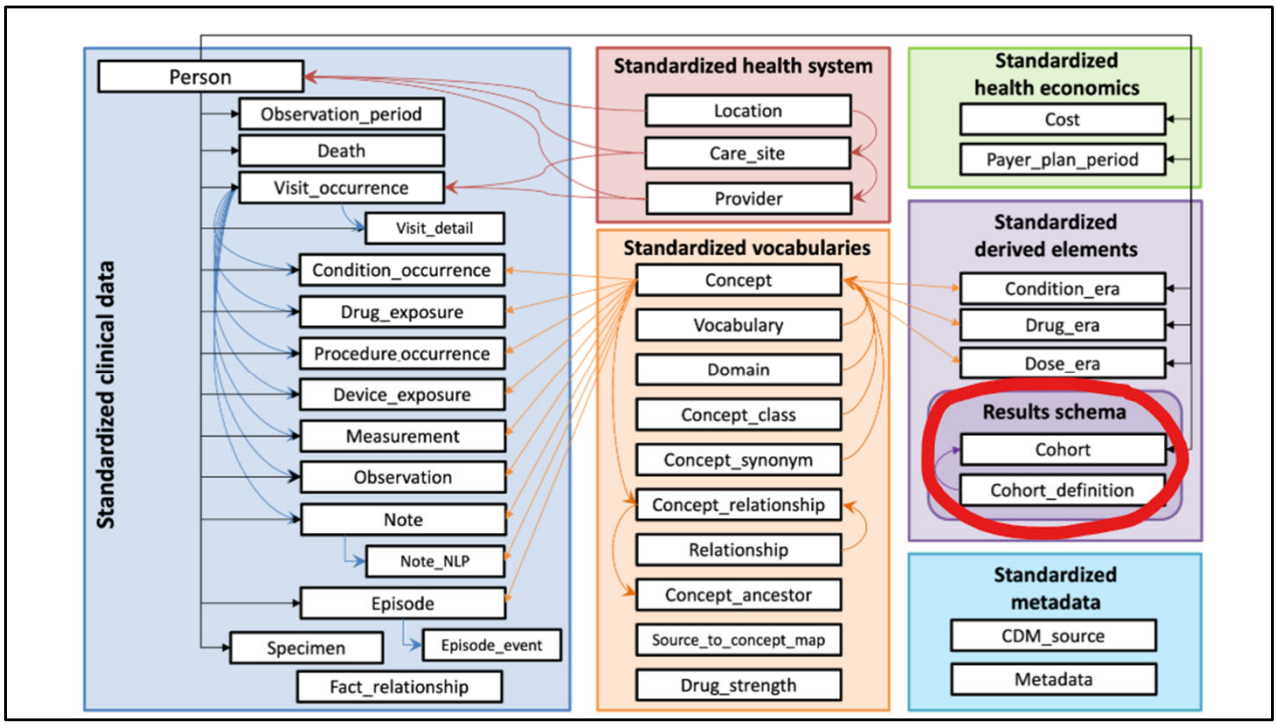
The OHDSI Mindset

Quick Review: The OMOP Common Data Model

- Standardized tables and fields
- Standardized vocabularies
- Relational database structure
- Shared across tools and institutions

You've already been introduced to the Common Data Model, but it's worth briefly revisiting.

The OMOP CDM defines a shared relational structure and vocabulary system. This is what allows different tools — and different institutions — to speak the same analytic language.



So far we have focused mostly on the clinical data tables on the left in blue, and the standardized vocabularies in orange. We have worked with the drug_era a few times when discussing cohort development and treatment pathway analysis. And believe it or not, you have already interacted with these circled tables: cohort and cohort definition. You probably just didn't realize it.

Where ATLAS Fits in the OHDSI Ecosystem

- ATLAS is a study *specification* tool
- Built directly on the OMOP Common Data Model
- Can produce no-code characterizations
- Shared foundation with all OHDSI analytics
- Bridge between design and analysis

Before we move further into analytics, I want to pause and explicitly connect ATLAS to everything that follows.

ATLAS is not a standalone application. It is a study specification tool that sits directly on top of the OMOP Common Data Model. Everything you do in ATLAS is grounded in the same data structures that the analytic libraries use.

ATLAS Uses the Same Tables as HADES

- PERSON, CONDITION_OCCURRENCE, DRUG_EXPOSURE, etc.
- No special ATLAS-only data layer
- Same database connection
- Same underlying records

One key point is that ATLAS does not use a separate copy of the data.

When you explore data or define cohorts in ATLAS, you are querying the same CDM tables that will later be accessed by HADES libraries and other analytic tools.

Cohort Definitions in ATLAS

- Graphical specification of inclusion criteria
- Saved as formal cohort definitions
- Independent of any specific analysis
- Reusable across studies

When you define cohorts in ATLAS you are creating formal, reusable cohort definitions.

These definitions are not tied to a specific model or outcome. They are design artifacts that can be reused across many types of analyses.

Cohort Generation and IDs

- Cohorts are materialized in the database
- Each cohort has a unique cohort ID
- Stored in standardized cohort tables
- IDs become references for analytics

Once cohorts are generated, they are stored in the database with unique cohort IDs.

Those IDs are critical. They are how analytic tools later refer to the cohorts you designed in ATLAS, without needing to redefine them.

How Analytics Use ATLAS Cohorts

- Analytics reference cohort IDs
- No re-specification of logic
- Same cohorts across tools
- Consistency by design

When you move into analytics, the cohort logic is not recreated.

Instead, analytic libraries simply reference the cohort IDs produced earlier. This ensures that the same population definition is used consistently across analyses.

I am going to walk you through an example of this soon.

Design Now, Analyze Later

- In the context of advanced analytics, ATLAS focuses on design
- Analytics (HADES/R) focuses on execution
- Clear separation of concerns
- Supports offline and large-scale analysis

This separation is intentional.

ATLAS is where study design happens. Analytics libraries are where execution happens. This allows you to design studies interactively, then run more advanced or large-scale analyses offline.

HADES “lives” in R but works hand in hand with Atlas

Nothing you have already learned is wasted

From ATLAS to Advanced Analytics

- ATLAS specifications become inputs
- HADES builds on these foundations
- No duplication of work
- HADES scales beyond the user interface

The key idea is that nothing you did in ATLAS is thrown away.

Those specifications become the foundation for more advanced analytics. HADES builds on them rather than replacing them.

Benefits

- Consistency across analyses
- Reduced rework
- Reproducible pipelines
- Easier collaboration

By connecting ATLAS and analytics in this way, OHDSI reduces duplication and inconsistency.

It also makes collaboration easier, because design decisions are explicit and reusable.

From ATLAS to Analytics

- Same data model
- Same cohorts
- Different layer of the stack
- Next: analytics beyond the UI

With this orientation in place, we can now move beyond ATLAS.

The next section focuses on analytics tools that operate on the same foundations, but allow more flexible, scalable, and programmable analyses.

Beyond Atlas: HADES and the OHDSI Analytics Ecosystem

Analytic use case	Type	Structure	Example
Clinical characterization	Disease Natural History	Amongst patients who are diagnosed with <insert your favorite disease>, what are the patient's characteristics from their medical history?	Amongst patients with rheumatoid arthritis, what are their demographics (age, gender), prior conditions, medications, and health service utilization behaviors?
	Treatment utilization	Amongst patients who have <insert your favorite disease>, which treatments were patients exposed to amongst <list of treatments for disease> and in which sequence?	Amongst patients with depression, which treatments were patients exposed to SSRI, SNRI, TCA, bupropion, esketamine and in which sequence?
	Outcome incidence	Amongst patients who are new users of <insert your favorite drug>, how many patients experienced <insert your favorite known adverse event from the drug profile> within <time horizon following exposure start>?	Amongst patients who are new users of methylphenidate, how many patients experienced psychosis within 1 year of initiating treatment?
Population-level effect estimation	Safety surveillance	Does exposure to <insert your favorite drug> increase the risk of experiencing <insert an adverse event> within <time horizon following exposure start>?	Does exposure to ACE inhibitor increase the risk of experiencing Angioedema within 1 month after exposure start?
	Comparative effectiveness	Does exposure to <insert your favorite drug> have a different risk of experiencing <insert any outcome (safety or benefit) > within <time horizon following exposure start>, relative to <insert your comparator treatment>?	Does exposure to ACE inhibitor have a different risk of experiencing acute myocardial infarction while on treatment, relative to thiazide diuretic?
Patient level prediction	Disease onset and progression	For a given patient who is diagnosed with <insert your favorite disease>, what is the probability that they will go on to have <another disease or related complication> within <time horizon from diagnosis>?	For a given patient who is newly diagnosed with atrial fibrillation, what is the probability that they will go onto to have ischemic stroke in next 3 years?
	Treatment response	For a given patient who is a new user of <insert your favorite chronically-used drug>, what is the probability that they will <insert desired effect> in <time window>?	For a given patient with T2DM who start on metformin , what is the probability that they will maintain HbA1C<6.5% after 3 years?
	Treatment safety	For a given patient who is a new user of <insert your favorite drug>, what is the probability that they will experience <insert adverse event > within <time horizon following exposure>?	For a given patients who is a new user of warfarin, what is the probability that they will have GI bleed in 1 year?

Speaker notes (for this slide)
This slide is meant to zoom out and answer a simple question:
What kinds of scientific questions does the OHDSI ecosystem actually support?
Across the top, you see different *analytic use cases*, and down the left you see three broad categories:
clinical characterization, **population-level effect estimation**, and **patient-level prediction**.

Note: I will be updating our GitHub training site with this table and other artifacts of this training for you to reference later.

What I want you to notice first is that none of these start with a method.
They all start with a *question structure*.

The green, red, blue, and yellow text is intentional.
These represent **replaceable components**:
a disease
a drug or exposure
an outcome
a time horizon

sometimes a comparator
And the key idea is this:
OHDSI tries to standardize the structure of questions, not just the answers.

Now, here's where this connects directly to what you already learned with ATLAS and Search.
When you defined cohorts — inclusion criteria, index events, time-at-risk logic — you were already specifying many of these building blocks:
who the patients are
when follow-up starts
what counts as exposure or outcome
ATLAS is where those definitions are made explicit and reusable.

This slide helps explain *why* that is important.
The same cohort definition you created in ATLAS can be reused for:
describing disease natural history
estimating population-level effects
or making patient-level predictions
You don't redefine the population each time.
You reuse it.

Another important thing to notice is the **separation of goals**.
Clinical characterization is largely descriptive.
Population-level effect estimation asks causal questions.
Patient-level prediction asks prognostic questions about individuals.
These are very different scientific tasks —
but in OHDSI, they are built on the **same data model and cohort infrastructure**.

This is where the ecosystem mindset comes in.
ATLAS handles *specification*:
cohorts, time windows, inclusion logic.
The analytics tools — which we'll talk about next — operate *offline*,
using those same specifications as inputs.
So rather than thinking "ATLAS versus HADES,"
it's more accurate to think **ATLAS first, analytics later**.

Finally, this slide is also meant to set expectations.
OHDSI is not a single method or a single package.
It's a framework for asking a family of related questions
in a consistent, reproducible way.
The rest of today is about how that framework is implemented in practice.

PLEASE NOTE:
None of this prevents you from asking novel questions.
It just asks you to express them in a way that others can inspect, reuse, and extend.
You're not expected to do all of these.
The value is that the *same foundation* supports whichever one you care about.

With that framing, we can now move from *what kinds of questions* OHDSI supports to *how the analytics ecosystem supports them* — beyond the ATLAS interface.

Speaker Notes: What is HADES?

HADES, previously known as the OHDSI Methods Library, is a collection of open-source R packages.

These packages are designed to work together to support a full observational study workflow.

Studies typically start with data in the OMOP Common Data Model and end with estimates, statistics, figures, and tables.

The packages interact directly with observational data in the CDM.

They can be used flexibly—either as lightweight tools to ensure cross-platform compatibility or as part of fully customized analyses.

HADES also offers advanced, standardized analytics for key use cases such as population characterization, population-level effect estimation, and patient-level prediction.

Importantly, HADES is built around best practices for observational research.

This includes an emphasis on transparency and reproducibility.

It also supports evaluating the operating characteristics of methods in a given context and applying empirical calibration to improve the reliability of study results.

HADES doesn't replace your knowlege

- You still need to understand the methods that you want to use
- Specify models accordingly
- You are still using R
- No mystery
- Methods are transparent and FAIR

As you will see in our upcoming examples, you still need to know how to specify your models, you still need to understand how the statistics works
If you would have done your analysis in R (probably), you are still using R
There is no black box, magic, or mystery taking place
In fact it is more transparent than you would otherwise have
What HADES allows you to do
Because the data are in a CDM, HADES allows us to pre-program everything – just drop in YOUR cohort IDs, concepts, etc.
Makes everything transparent and reproducible
Allows you to share with others
Build upon "skeleton" code
Share your work internally and within a network
Follow FAIR data principles, regulatory guidance, auditing, transparency, sharing code etc

Skills you may need to improve

- Specifying your cohorts properly
- Relational databases
- Understanding how the CDM works (Cohort tables, cohort ID#)
- Understanding R, GitHub basics
- Understanding how open source software development works
  - You can contribute, report bugs
  - Ask for help in the forums
- Connecting to your OMOP instance while preserving privacy
- Understanding the connection between Atlas and R code

HADES: Analytics Packages Overview

- HADES is a modular analytics ecosystem
- Packages have clear, complementary roles
- Built on ATLAS cohorts and the OMOP CDM
- Designed for scalable, reproducible research

This section orients you to the HADES ecosystem at a high level. The goal is not to teach each package in detail, but to understand what exists, what problems each package solves, and how they fit together on top of ATLAS and the CDM.

Full disclosure: I have not worked directly with all of these packages, although I have worked with many and am familiar with some of the methods involved from non-OHDSI projects.

CohortMethod

- Population-level causal inference
- New-user cohort designs
- Propensity score & outcome models
- Comparative effectiveness & safety

CohortMethod is used for population-level effect estimation. It implements new-user cohort designs and relies on cohorts defined earlier in ATLAS.

Incidentally I am just starting to explore this in the context of an external controls project.

What "new-user" means

In CohortMethod, a *new user* is a patient who:

Has **no prior exposure** to the target drug (or drug class) during a defined **washout period**

Initiates the drug at a clearly defined **index date**

Is then followed forward in time for outcomes

This mirrors the logic of a randomized controlled trial, where treatment starts at time zero.

Self-Controlled Case Series (SCCS)

- Within-person design
- Controls for time-invariant confounding
- Splines for age and seasonality
- Safety surveillance

SCCS implements a classic within-person design, especially useful for safety questions.

Self-Controlled Cohort

- Pre-exposure time as control
- Simpler within-person design
- Scalable for large screening studies

This design is computationally simpler than SCCS and often used for exploratory analyses.

Case-Control

- Matched case–control studies
- Matching on demographics and visit date
- Can nest within cohorts

Case-control supports traditional epidemiologic designs, especially for rare outcomes.

Case-Crossover

- Transient exposures
- Within-person comparison
- Supports case–time–control

Case-crossover is used when exposure effects are acute and time-varying.

PatientLevelPrediction

- Individual risk prediction
- Machine learning & regression
- Model performance & calibration

PatientLevelPrediction focuses on prognosis rather than causation.

EmpiricalCalibration

- Bias calibration using negative controls
- Adjusts confidence intervals and p-values
- Improves interpretability

EmpiricalCalibration quantifies and adjusts for residual bias in observational studies.

MethodEvaluation

- Benchmarking analytic methods
- Uses reference sets and injected signals
- Supports method development

MethodEvaluation helps understand when methods work well and when they fail.

EvidenceSynthesis

- Combine results across databases
- Uses diagnostics and estimates
- Supports federated research

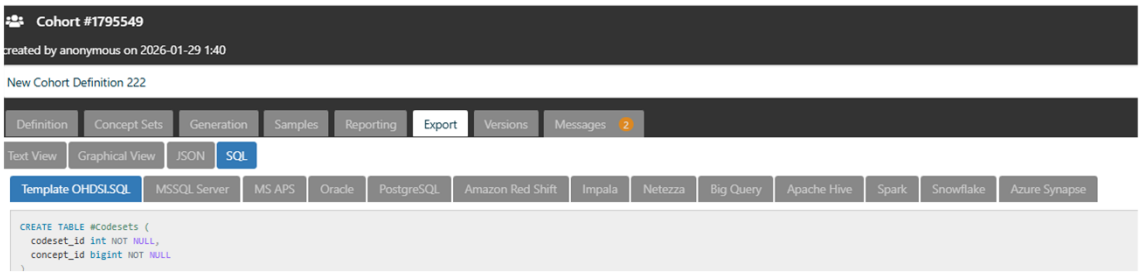
EvidenceSynthesis summarizes results across sites in network studies.

DatabaseConnector

- Unified database access
- Supports multiple SQL platforms
- Used by most HADES packages

DatabaseConnector manages database connections across platforms.

SqlRender



SqlRender ensures SQL works across different database technologies.
This is very cool when you are trying to use someone else's SQL code – remember when we export our cohort definition from Atlas, we have options of different SQL dialects?

Cyclops

- High-performance regression engine
- Logistic, Poisson, Cox models
- Regularization for large-scale data

Cyclops provides scalable statistical modeling used by many HADES packages.

FeatureExtraction

- Automated covariate generation
- Uses CDM data
- Supports prediction & adjustment

FeatureExtraction automates creation of large feature sets.

ParallelLogger

- Parallel computation
- Logging and monitoring
- Supports long-running studies

ParallelLogger helps manage large analyses reliably.

R Concepts Helpful for Working with HADES

- renv for reproducible R environments
- Environment variables for credentials
- Keyring / secrets management
- DatabaseConnector for secure connections to your CDM
- File-based study outputs
- GitHub for downloading packages, version control, sharing, and issue reporting

Since so much of HADES is simply “paint by numbers” with so many skeleton templates, you don’t need to be an R expert, but some specific skills make working with HADES much smoother. renv helps lock package versions for reproducibility. Database credentials should never be hard-coded; use environment variables or keyring-based tools instead.

Most OHDSI studies produce file-based outputs that can be archived and shared alongside code.

In OHDSI, results don’t just appear on screen — they are written to disk in a structured way so they can be shared, reviewed, combined, and reused.

File-based outputs in OHDSI
Structured folders, not ad hoc files
Human-readable + machine-readable formats
Designed for sharing and reuse
Support federated and FAIR workflows

OHDSI as Collective Experience at Scale

- Community built around large EHR & claims data
- Best practices *emerged* from repeated use, not theory alone
- Practices encoded in tools, workflows, and outputs
- File-based results enable reproducibility, sharing, and synthesis
- Methods continue to evolve through open use, critique, regulatory landscape, emerging use cases, etc

I want to be very clear about how I'm framing OHDSI here, especially for a methodologically skeptical audience. And if you are skeptical,you are right to be skeptical – what kind of scientist would you be if you weren't?

OHDSI is not claiming to be the only correct way to do epidemiology, and it's not proposing new methods by decree.

Instead, it represents a community that has spent years working with large EHR and claims databases and has gradually converged on a set of practices that *work at scale*.

Many of the design choices you see — file-based outputs, standardized cohorts, versioned code, federated execution —
didn't arise from abstract theory alone.
They arose because people repeatedly ran into the same problems:
Non-reproducible results
analyses that didn't port to new databases
difficulty combining evidence across sites
results that couldn't be audited or revisited later

And these people are often drug company epi folks – who have a lot of money and their reputation on the line, not to mention regulators watching every move - to get this right

Those lessons are now encoded directly into tools and workflows rather than living only in papers or institutional memory.
File-based outputs are a good example of this.
They make results portable, inspectable, and reusable — which turns out to be essential once studies involve multiple databases or even internal collaborators.
You will see that with HADES these portable, reproducible tools extend to study results, too. With Rshiny app viewers and lots of great ways to display your results which you probably otherwise might struggle to develop.

Importantly, none of this freezes methods in place.
OHDSI tools and practices continue to evolve through open use, critique, and community discussion.
In that sense, the ecosystem is less a prescription and more a *living record* of what has proven workable for large-scale observational research.

So a useful way to think about OHDSI is not as "the right method,"
but as *collective experience made operational* — in code, structure, and shared expectations.

Optional closing line (if you want to transition)
With that framing, the next question becomes:
How does this experience actually show up when we run analyses?

OHDSI Uses Methods You Already Know

- Built on standard epidemiologic models (logistic, Cox, Poisson)
- Uses established machine-learning approaches for prediction
- Novelty is **infrastructure and workflow**, not new mathematics
- Code, assumptions, and diagnostics are fully inspectable
- Designed to scale familiar methods to large EHR & claims data

This slide addresses an important and reasonable concern:
"If I trust the tools I already use in R for epidemiology, why should I trust OHDSI?"
The key point is that OHDSI does **not** introduce a new statistical paradigm.
Under the hood, most OHDSI analyses rely on models you already know:
logistic regression, Cox proportional hazards models, and Poisson models.
If you trust glm() and coxph(), you already trust the mathematical foundations of OHDSI.
What I find interesting sometimes about OHDSI skeptics is that I don't think I have ever heard someone ask how we know SAS or SPSS tools are accurate or calibrated correctly. And yet OHDSI is transparent about every single software tool.

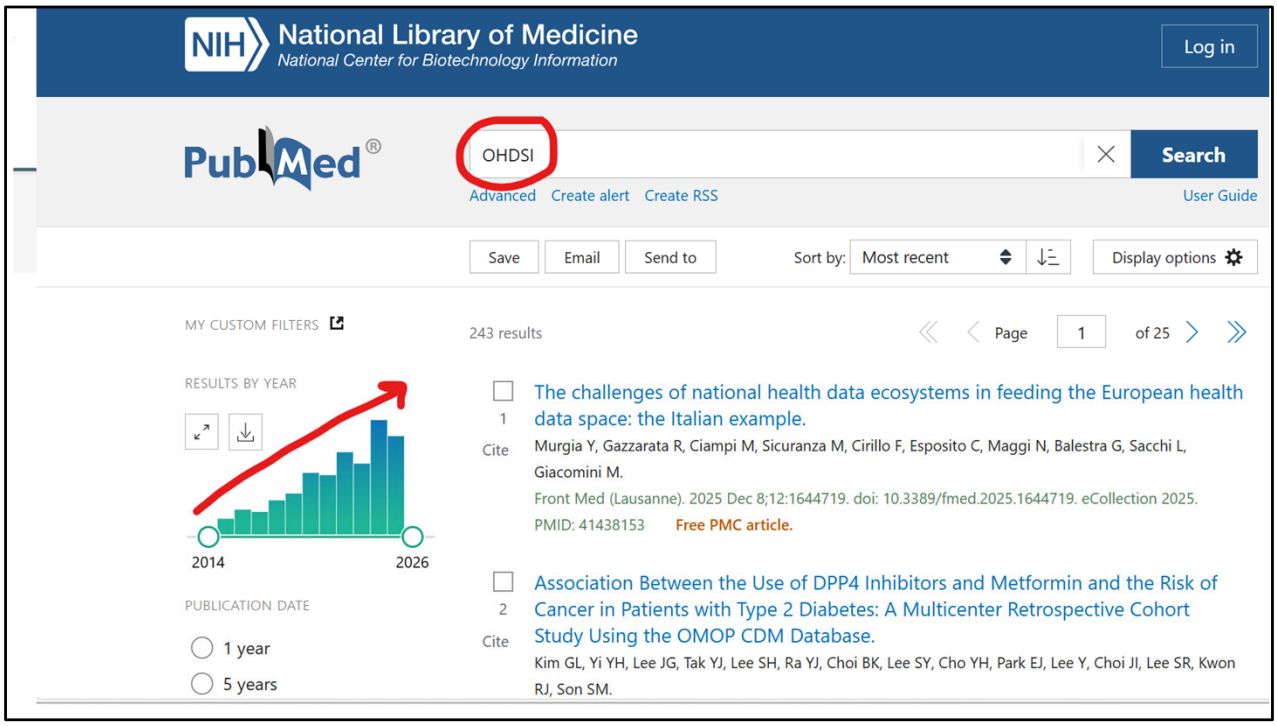
Where OHDSI differs is not in inventing new math, but in how those methods are **operationalized**.
The tools are engineered to handle:
very large datasets
high-dimensional covariates
repeated execution across databases
reproducible, auditable workflows
In other words, the innovation is *infrastructure*, not statistical novelty.

This also helps address the “black box” concern.
OHDSI tools are not black boxes in the usual sense:
the code is open
model settings are explicit
diagnostics are generated by default
intermediate outputs are written to disk
You can inspect every step — often more easily than in bespoke, one-off analysis scripts.

A useful way to think about OHDSI is this:
it takes methods epidemiologists already trust and places them in a framework designed to survive scale, replication, and scrutiny.
That is why these tools are increasingly used not just in academia, but also in regulatory and multi-institutional research settings.

OHDSI tools are opinionated, but they are not opaque — they make assumptions more visible, not less.

.

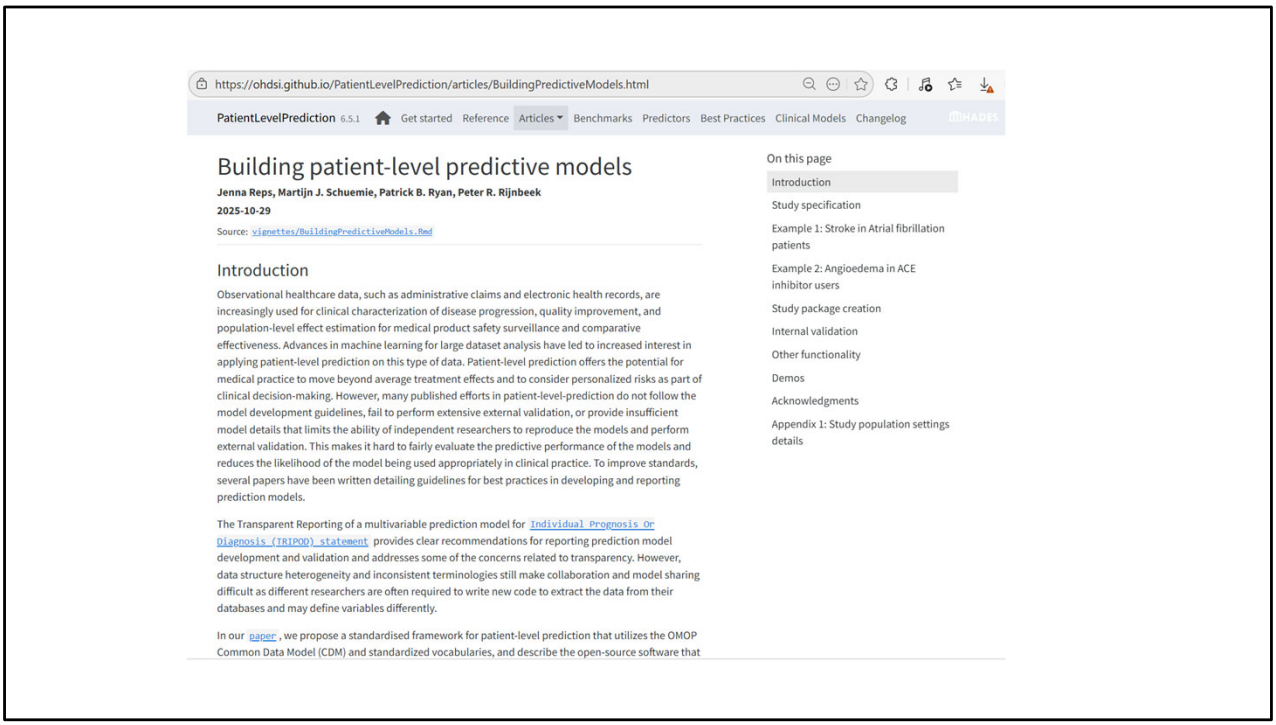

National Library of Medicine
National Center for Biotechnology Information
Log in
PubMed®
OHDSI
Search
Advanced
Create alert
Create RSS
User Guide
Save
Email
Send to
Sort by: Most recent
Display options
MY CUSTOM FILTERS
243 results
Page 1 of 25
RESULTS BY YEAR
2014
2026
PUBLICATION DATE
1 year
5 years
1
Cite
The challenges of national health data ecosystems in feeding the European health data space: the Italian example.
Murgia Y, Gazzarata R, Ciampi M, Sicuranza M, Cirillo F, Esposito C, Maggi N, Balestra G, Sacchi L, Giacomini M.
Front Med (Lausanne). 2025 Dec 8;12:1644719. doi: 10.3389/fmed.2025.1644719. eCollection 2025.
PMID: 41438153
Free PMC article.
2
Cite
Association Between the Use of DPP4 Inhibitors and Metformin and the Risk of Cancer in Patients with Type 2 Diabetes: A Multicenter Retrospective Cohort Study Using the OMOP CDM Database.
Kim GL, Yi YH, Lee JG, Tak YJ, Lee SH, Ra YJ, Choi BK, Lee SY, Cho YH, Park EJ, Lee Y, Choi JI, Lee SR, Kwon RJ, Son SM.

Key Takeaways

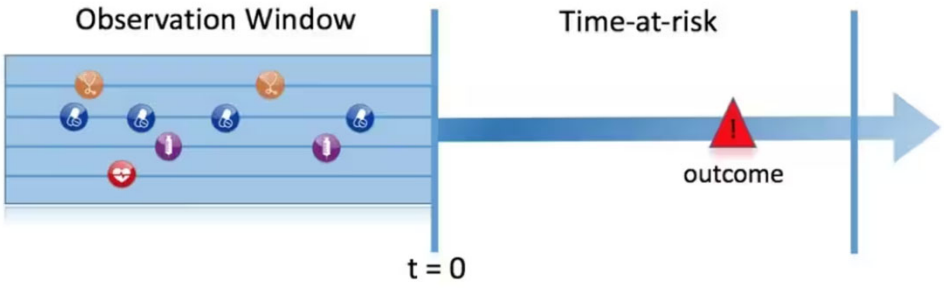
- ATLAS defines study design
- HADES executes analytics offline
- Packages are modular and interoperable
- Infrastructure supports FAIR science

The most important takeaway is how the pieces fit together. You are not learning isolated tools, but a coherent analytics stack.

Hands-on: Patient-level Prediction



First I want to share that there is no shortage of documentation for any of these libraries. Quite the opposite. When you google this package you will find youtube videos and github repos and posters and abstracts and articles. So my hope here is to give you a taste of what you will receive.

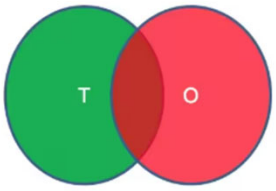


PatientLevelPrediction is an R package for building and validating patient-level predictive models using data in the OMOP Common Data Model format.
This package is described in the paper by Reps JM, Schuemie MJ, Suchard MA, Ryan PB, Rijnbeek PR. Design and implementation of a standardized framework to generate and evaluate patient-level prediction models using observational healthcare data. J Am Med Inform Assoc. 2018;25(8):969-975. that I shared before.

This figure illustrates the prediction problem we address. Among a population at risk, we aim to predict which patients at a defined moment in time (t = 0) will experience some outcome during a time-at-risk. Prediction is done using only information about the patients in an observation window prior to that moment in time.

Input parameter	Design choice
Target cohort (T)	
Outcome cohort (O)	
Time-at-risk	
Model development -which algorithm(s)? -which parameters? -which covariates?	

We extract data for the patients in the Target Cohort (T) of which some will experience the outcome (O) in T



The prediction problem

As shown on this slide, to define a prediction problem we have to define t=0 by a Target Cohort (T), the outcome we like to predict by an outcome cohort (O), and the time-at-risk (TAR). Furthermore, we have to make design choices for the model we like to develop, and determine the observational datasets to perform internal and external validation. This conceptual framework works for all type of prediction problems, for example those presented in Figure 3.

Type	Structure	Example
Disease onset and progression	Amongst patients who are newly diagnosed with <insert your favorite disease>, which patients will go on to have <another disease or related complication> within <time horizon from diagnosis>?	Among newly diagnosed AFib patients, which will go onto to have ischemic stroke in next 3 years?
Treatment choice	Amongst patients with <indicated disease> who are treated with either <treatment 1> or <treatment 2>, which patients were treated with <treatment 1> (on day 0)?	Among AFib patients who took either warfarin or rivaroxaban, which patients got warfarin? (as defined for propensity score model)
Treatment response	Amongst patients who are new users of <insert your favorite chronically-used drug>, which patients will <insert desired effect> in <time window>?	Which patients with T2DM who start on metformin stay on metformin after 3 years?
Treatment safety	Amongst patients who are new users of <insert your favorite drug>, which patients will experience <insert your favorite known adverse event from the drug profile> within <time horizon following exposure start>?	Among new users of warfarin, which patients will have GI bleed in 1 year?
Treatment adherence	Amongst patients who are new users of <insert your favorite chronically-used drug>, which patients will achieve <adherence metric threshold> at <time horizon>?	Which patients with T2DM who start on metformin achieve >=80% proportion of days covered at 1 year?

To define a prediction problem we have to define t=0 by a Target Cohort (T), the outcome we like to predict by an outcome cohort (O), and the time-at-risk (TAR). Furthermore, we have to make design choices for the model we like to develop, and determine the observational datasets to perform internal and external validation. This conceptual framework works for all type of prediction problems, for example those presented below (T=green, O=red).

Features of Patient-level Prediction

- Takes one or more target cohorts (Ts) and one or more outcome cohorts (Os) and develops and validates models for all T and O combinations.
- Allows for multiple prediction design options.
- Extracts the necessary data from a database in OMOP Common Data Model format for multiple covariate settings.
- Uses a large set of covariates including for example all drugs, diagnoses, procedures, as well as age, comorbidity indexes, and custom covariates.
- Allows you to add custom covariates or cohort covariates.
- Includes a large number of state-of-the-art machine learning algorithms that can be used to develop predictive models, including Regularized logistic regression, Random forest, Gradient boosting machines, Decision tree, Naive Bayes, K-nearest neighbours, Neural network, AdaBoost and Support vector machines.

Features of Patient-level Prediction

- Allows you to add custom feature engineering
- Allows you to add custom under/over sampling (or any other sampling) [note: based on existing research this is not recommended]
- Contains functionality to externally validate models.
- Includes functions to plot and explore model performance (ROC + Calibration).
- Build ensemble models using EnsemblePatientLevelPrediction.
- Build Deep Learning models using DeepPatientLevelPrediction.
- Generates learning curves.
- Includes a shiny app to interactively view and explore results.
- In the shiny app you can create a html file document (report or protocol) containing all the study results.

Technology

- PatientLevelPrediction is an R package, with some functions using python through reticulate.
- System Requirements
  - R (version 4.0 or higher). Installation on Windows requires RTools.
  - Libraries used in PatientLevelPrediction require Java and Python.
  - The python installation is required for some of the machine learning algorithms.
  - Recommended: Python 3.9 or higher using Anaconda (https://www.continuum.io/downloads).

Patient-Level Prediction in OHDSI: Big Picture

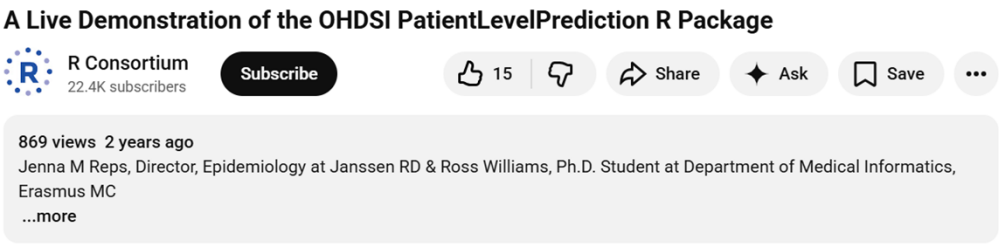
- Predict risk for individual patients
- Requires explicit study specification
- Same principles as traditional epidemiology
- Example: disease onset and progression

Patient-level prediction in OHDSI focuses on prognosis rather than causality. The study must be clearly specified up front, including the population, outcome, time horizon, and evaluation strategy.

Prediction Type: Disease Onset and Progression

- Predict future disease occurrence
- Defined relative to an index event
- Focus on risk over time

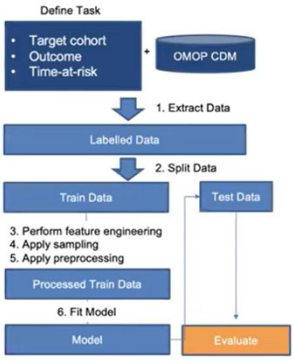
Disease onset and progression prediction asks what is likely to happen next given a clearly defined starting point.



https://youtu.be/qcPMf5Kq3Xw?si=VT8rhpvEe9r2IcRP

The following example comes from Jenna Reps' work. She is a great leader in the OHDSI community and literally wrote the paper on PatientLevelPrediction.

Flexible And Standardized Prediction Pipeline



The Pipeline

First you need a clearly defined clinically useful prediction task (T,O and TAR) and an OMOP CDM database.

Then the PatientLevelPrediction R package can extract the data, learn the model and validate the model. You just need to specify the model design components.

R Package: https://github.com/OHDSI/PatientLevelPrediction

OHDSI

What I amabout to show you comes from Jenna Repp's presentation at R Consorium

Step-By-Step Example

A demo of the package for the following prediction task:

- Target Population (T): New users of lisinopril with prior hypertension
- Outcome (O): Angioedema
- Time-at-risk (TAR): 1 day to 365 days after starting lisinopril

This first example I am going to show you builds the cohorts in Atlas and does everything else including all model settings and covariates and analysis in R/HADES. The next example will show how to build the cohorts in Atlas OR SQL/R and specify the settings within Atlas, then run the analysis in R/HADES.
You MUST run the final analysis in R. But there's a lot you can do either in R or Atlas depending on your preference.

Set-up

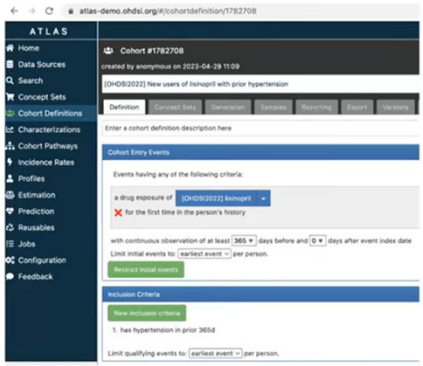
Requirements: You need Java (as we use JDBC to connect to the OMOP database) and R. Some parts of the package require python but you can use reticulate to install mini-conda.

First you need to install the PatientLevelPrediction R package:

```r
install.packages("remotes")
remotes::install_github("OHDSI/PatientLevelPrediction")

# to setup python (optional)
library(PatientLevelPrediction)
reticulate::install_miniconda()
configurePython(envname='r-reticulate', envtype='conda')
```

Create Target Population In ATLAS

Create a cohort in ATLAS (public ATLAS is https://atlas-demo.ohdsi.org) for target population (take note of the id)

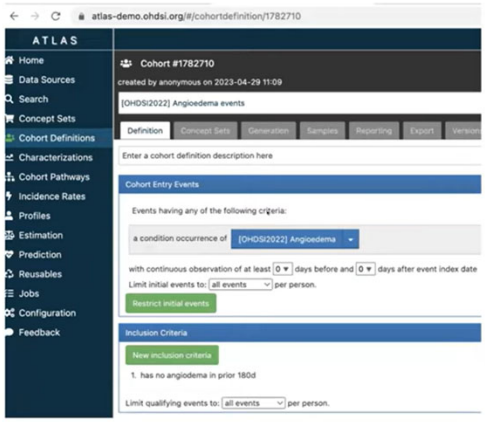


Target: https://atlas-demo.ohdsi.org/#/cohortdefinition/1782708

R Package: https://github.com/OHDSI/PatientLevelPrediction

Create Outcome In ATLAS

Create a cohort in ATLAS for outcome (take note of the id)



Outcome: https://atlas-demo.ohdsi.org/#/cohortdefinition/1782710

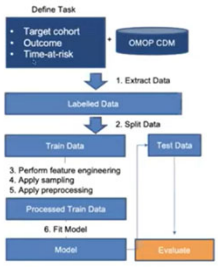
Define Time-at-risk And Inclusions

Specify when to predict the outcome relative to target cohort start/end plus any additional inclusions:

```
library(PatientLevelPrediction)
populationSettings <- createStudyPopulationSettings(
    firstExposureOnly = T,
    removeSubjectsWithPriorOutcome = T,
    priorOutcomeLookback = 180,
    riskWindowStart = 1,
    startAnchor = 'cohort start',
    riskWindowEnd = 365,
    endAnchor = 'cohort start',
    requireTimeAtRisk = F
    )
```

We predict the outcome in the 1-year after target population cohort start.

1. Feature Extraction Settings



The Pipeline

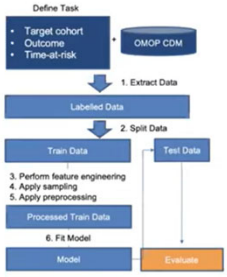
Choose features you want to use in the model. We have standardized features to pick from or you can write custom SQL.

```
library(FeatureExtraction)
covariateSettings <- createCovariateSettings(
    useDemographicsGender = T,
    useDemographicsAgeGroup = T,
    useConditionGroupEraLongTerm = T,
    useDrugGroupEraLongTerm = T
    )
```

R Package: https://github.com/OHDSI/PatientLevelPrediction

OHDSI

2. Split Settings

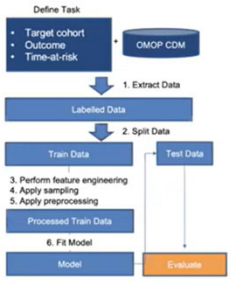


Choose the test/validation/train options.

```
library(PatientLevelPrediction)
splitSettings <- createDefaultSplitSetting(
  testFraction = 0.25,
  splitSeed = 1535,
  nfold = 3,
  type = 'stratified' #subject/time
    )
```

The Pipeline

3. Feature Engineering Settings

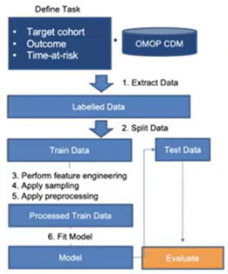


Add code to perform any feature engineering/selection to the covariates.

```
library(PatientLevelPrediction)
# we currently do not have many
# options for feature engineering
featureEngineering <- createFeatureEngineeringSet
  type = 'none'
  )
```

The Pipeline

4. Sample Settings



Select none/under-sample/over-sample to address class imbalance [Note: research shows this is not a good idea].

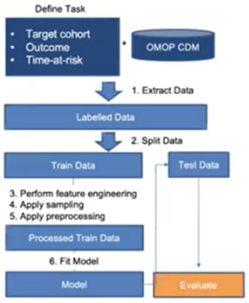
```
library(PatientLevelPrediction)
sampleSettings <- createSampleSettings(
  type = 'none'
  )
```

The Pipeline

R Package: https://github.com/OHDSI/PatientLevelPrediction

OHDSI

5. Preprocess Settings

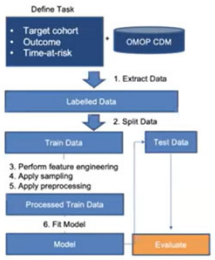


Options to remove redundant covariates, normalize covariate values and remove infrequently observed covariates.

```
library(PatientLevelPrediction)
preprocessSettings <- createPreprocessSettings(
    minFraction = 1/10000,
    normalize = T,
    removeRedundancy = T
  )
```

The Pipeline

6. Model Fitting Settings



The Pipeline

Pick the classifier from the library or create your own – also specify the hyper-parameters and search strategy.

```r
library(PatientLevelPrediction)
# Option 1: Logistic regression
modelLR <- setLassoLogisticRegression()

# Option 2: Gradient Boosting Machines
modelGBM <- setGradientBoostingMachine(
  maxDepth = c(2,4,10,17)
  )

# Option 3: Random Forest
modelRF <- setRandomForest(
  mtries = list(1000),
  maxDepth = list(2,4,10,17)
    )

# we have many options...
```

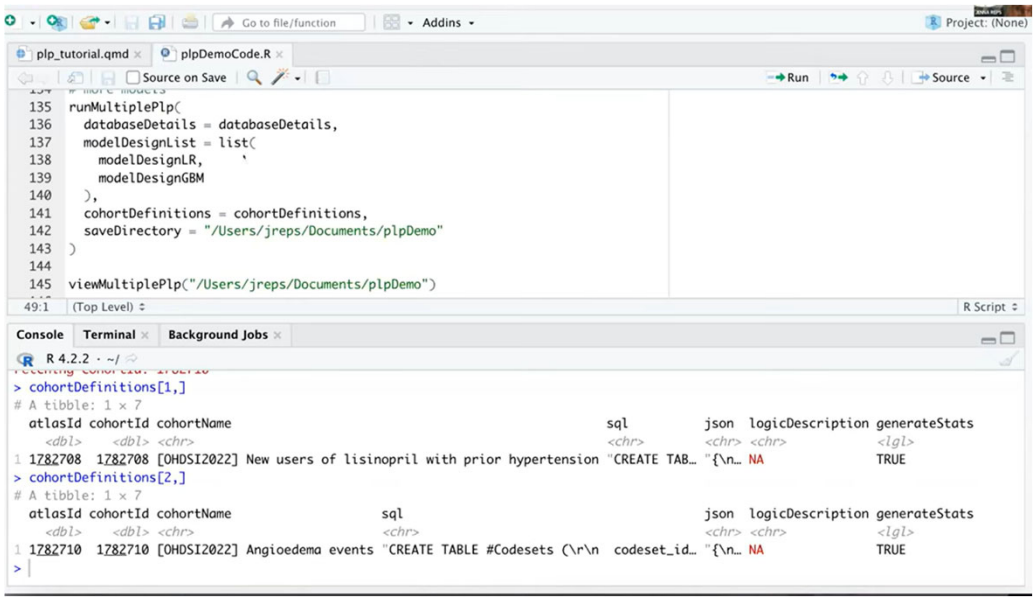
R Package: https://github.com/OHDSI/PatientLevelPrediction

OHDS

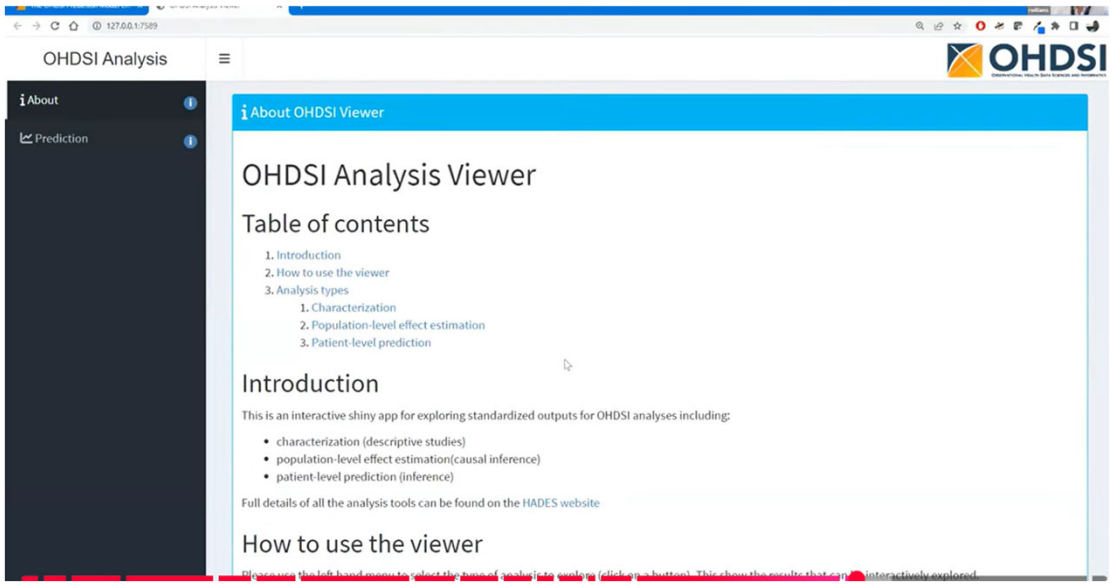
Full Model Design

The prediction task components (T,O & TAR) plus the model design components:

```
library(PatientLevelPrediction)
modelDesignLR <- createModelDesign(
  targetId = 1782708, # ATLAS ID
  outcomeId = 1782710, # ATLAS ID
  populationSettings = populationSettings,
  covariateSettings = covariateSettings,
  preprocessSettings = preprocessSettings,
  splitSettings = splitSettings,
  modelSettings = modelLR
  )
```



I won't go through every single step but wanted to show you a snapshot here of what the analysis actually looks like in R.



Once we execute the study, the runPlp() function returns the trained model and the evaluation of the model on the train/test sets.
You can interactively view the results by running: viewPlp(runPlp=lrResults). This will generate a Shiny App in your browser in which you can view all performance measures created by the framework.
Now I am going to show you what that looks like in more detail.

PatientLevelPrediction Explorer | Internal Validation | External Validation

Evaluation Summary | Characterization | ROC | Calibration | Demographics | Preference | Box Plot | Settings

Evaluation Summary

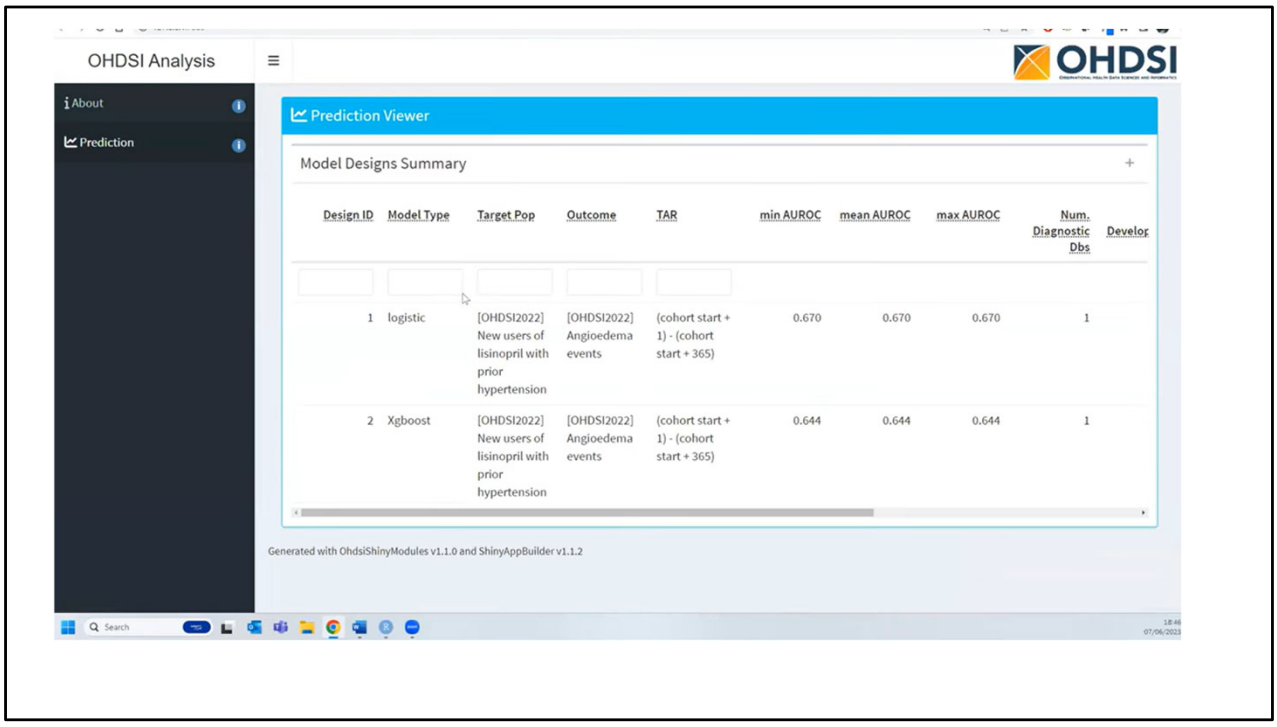
Show 25 entries Search:

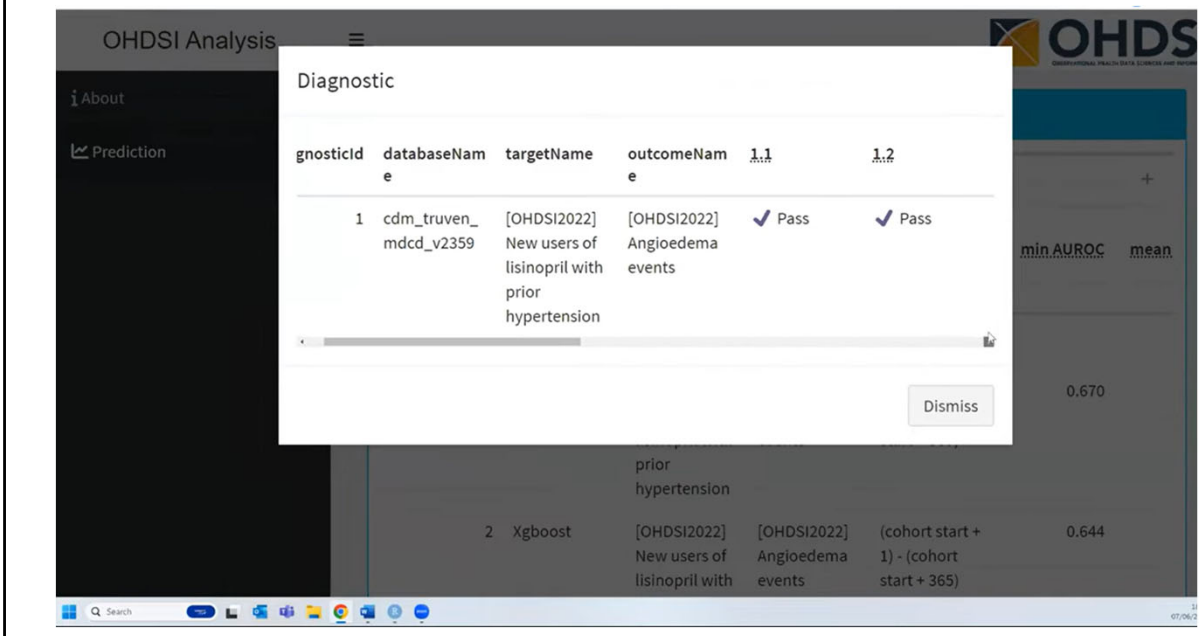
	Metric	test	train
1	AUC	0.72130	0.75348
2	AUC_lb95ci	0.70057	0.74215
3	AUC_ub95ci	0.74203	0.76482
4	AUPRC	0.10971	0.13571
5	BrierScaled	0.03755	0.04902
6	BrierScore	0.03355	0.03304
7	CalibrationIntercept.Intercept	-0.00089	-0.00813
8	CalibrationSlope.Gradient	1.02041	1.22457
9	outcomeCount	601.00000	1802.00000
10	populationSize	16685.00000	50054.00000
11	Incidence	3.60204	3.60011

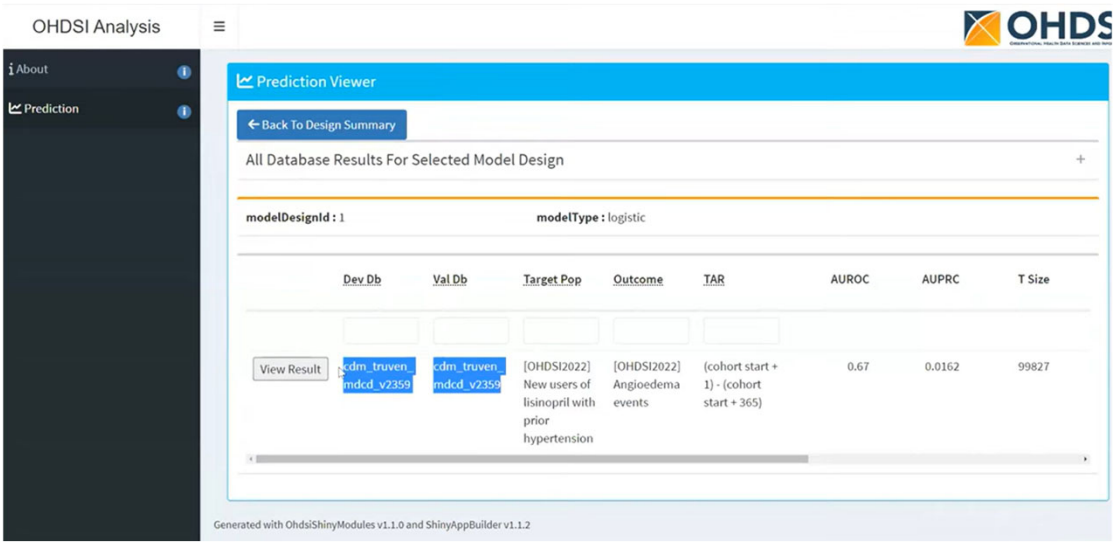
Showing 1 to 11 of 11 entries Previous 1 Next

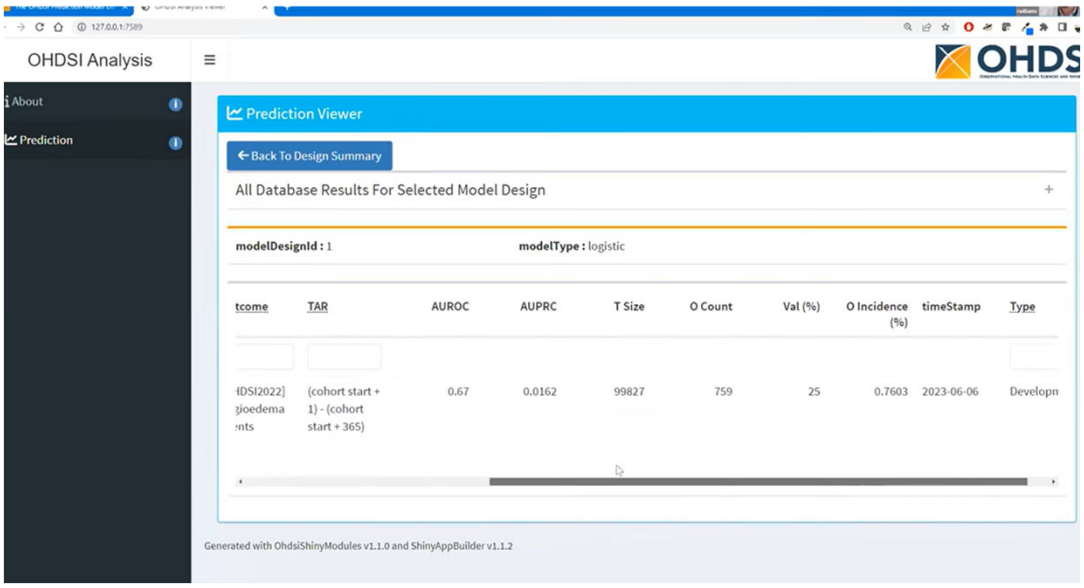
Summary of all the performance measures of the analyses

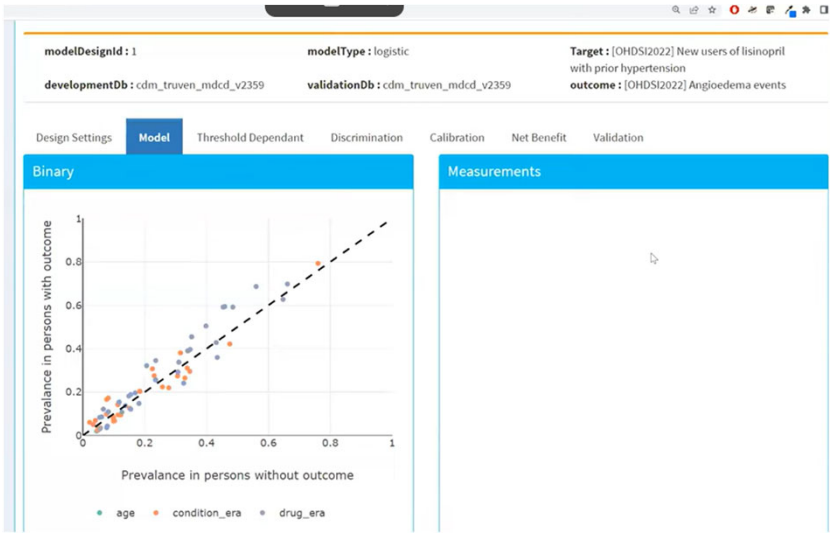
Furthermore, many interactive plots are available in the Shiny App, for example the ROC curve in which you can move over the plot to see the threshold and the corresponding sensitivity and specificity values.

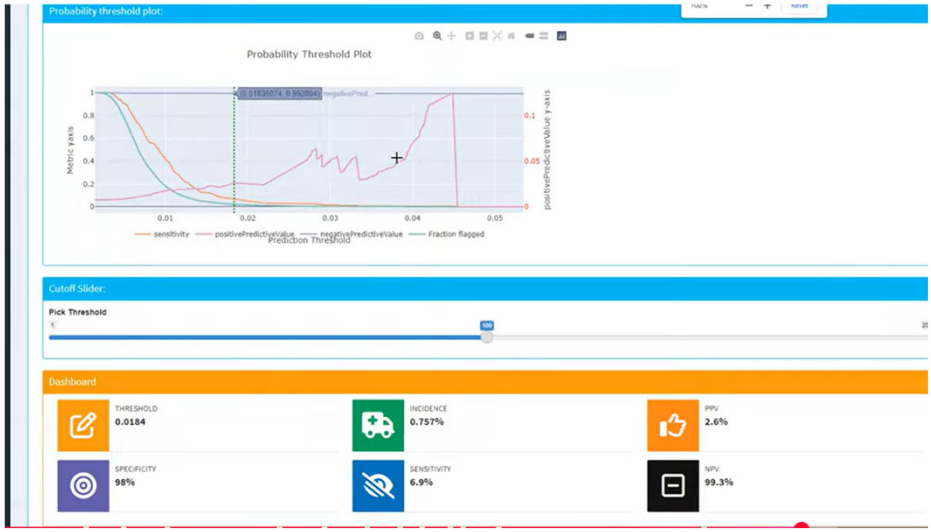

OHDSI Analysis
OHDSI
About
Prediction
Prediction Viewer
Model Designs Summary
Design ID
Model Type
Target Pop
Outcome
TAR
min AUROC
mean AUROC
max AUROC
Num. Diagnostic Dbs
Develop
1
logistic
[OHDSI2022] New users of lisinopril with prior hypertension
[OHDSI2022] Angioedema events
(cohort start + 1) - (cohort start + 365)
0.670
0.670
0.670
1
2
Xgboost
[OHDSI2022] New users of lisinopril with prior hypertension
[OHDSI2022] Angioedema events
(cohort start + 1) - (cohort start + 365)
0.644
0.644
0.644
1
Generated with OhdsiShinyModules v1.1.0 and ShinyAppBuilder v1.1.2
Search

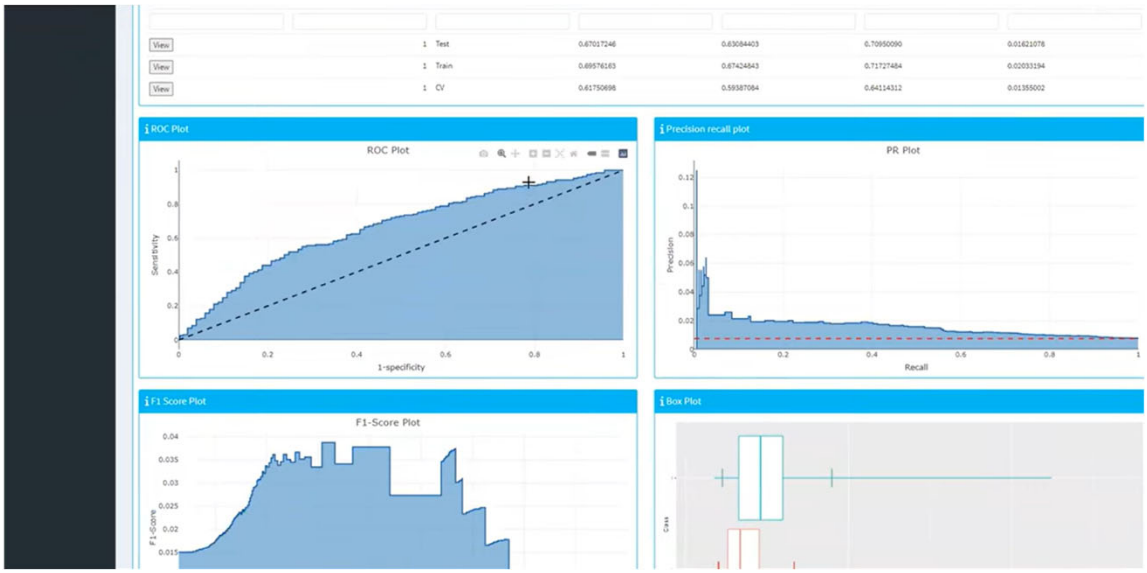

OHDSI Analysis
About
Prediction
Diagnostic
gnosticId
databaseNam e
targetName
outcomeNam e
1.1
1.2
1
cdm_truven_ mdcd_v2359
[OHDSI2022] New users of lisinopril with prior hypertension
[OHDSI2022] Angioedema events
Pass
Pass
Dismiss
min AUROC
mean
0.670
prior hypertension
2
Xgboost
[OHDSI2022] New users of lisinopril with
[OHDSI2022] Angioedema events
(cohort start + 1) - (cohort start + 365)
0.644

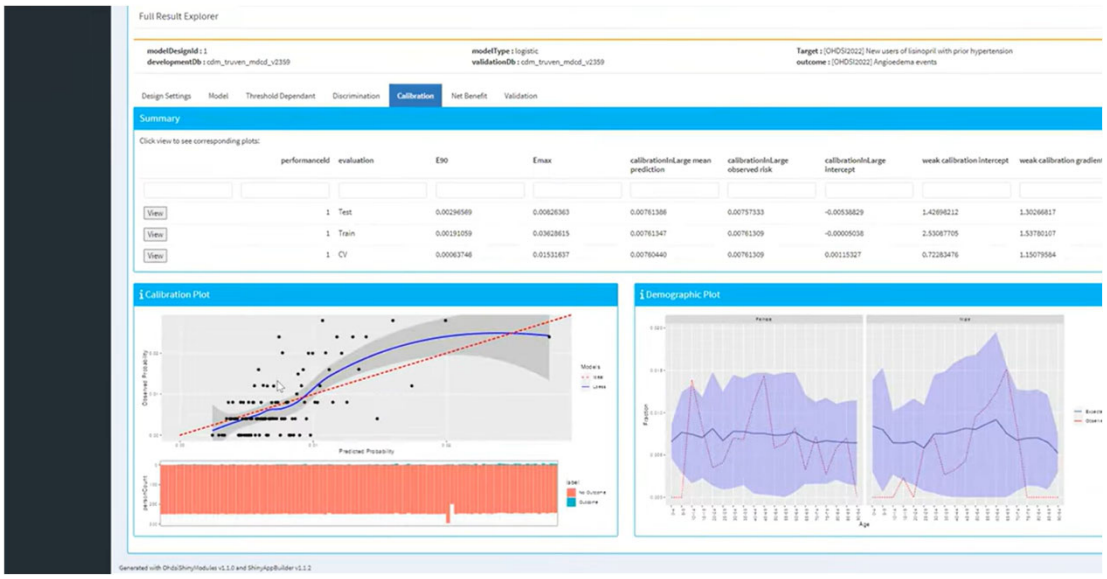

OHDSI Analysis
About
Prediction
Prediction Viewer
Back To Design Summary
All Database Results For Selected Model Design
modelDesignId : 1
modelType : logistic
Dev Db
Val Db
Target Pop
Outcome
TAR
AUROC
AUPRC
T Size
View Result
cdm_truven_mdcd_v2359
cdm_truven_mdcd_v2359
[OHDSI2022] New users of lisinopril with prior hypertension
[OHDSI2022] Angioedema events
(cohort start + 1) - (cohort start + 365)
0.67
0.0162
99827
Generated with OhdsiShinyModules v1.1.0 and ShinyAppBuilder v1.1.2


OHDSI Analysis
About
Prediction
Prediction Viewer
Back To Design Summary
All Database Results For Selected Model Design
modelDesignId : 1
modelType : logistic
tcome
TAR
AUROC
AUPRC
T Size
O Count
Val (%)
O Incidence (%)
timeStamp
Type
HDSI2022] gioedema nts
(cohort start + 1) - (cohort start + 365)
0.67
0.0162
99827
759
25
0.7603
2023-06-06
Developn
Generated with OhdsiShinyModules v1.1.0 and ShinyAppBuilder v1.1.2

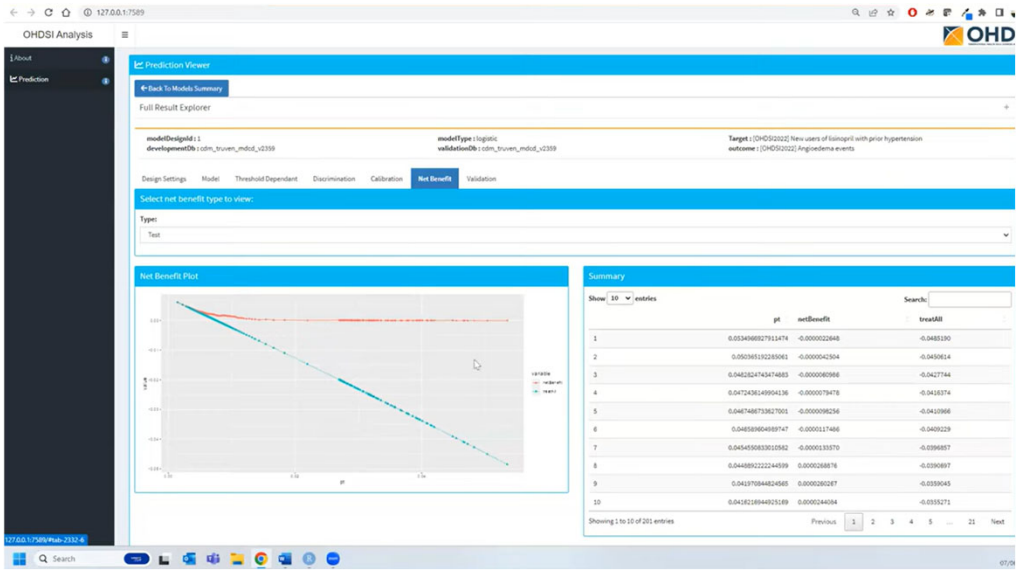

modelDesignId : 1
modelType : logistic
Target : [OHDSI2022] New users of lisinopril with prior hypertension
developmentDb : cdm_truven_mdcd_v2359
validationDb : cdm_truven_mdcd_v2359
outcome : [OHDSI2022] Angioedema events
Design Settings
Model
Threshold Dependant
Discrimination
Calibration
Net Benefit
Validation
Binary
Measurements
Prevalance in persons with outcome
Prevalance in persons without outcome
0
0.2
0.4
0.6
0.8
1
age
condition_era
drug_era


Probability threshold plot:
Probability Threshold Plot
Metric yaxis
positivePredictiveValue y-axis
Prediction Threshold
sensitivity
positivePredictiveValue
negativePredictiveValue
Fraction flagged
Cutoff Slider:
Pick Threshold
Dashboard
THRESHOLD
0.0184
INCIDENCE
0.757%
PPV
2.6%
SPECIFICITY
98%
SENSITIVITY
6.9%
NPV
99.3%


View 1 Test 0.67017246 0.63084403 0.70950090 0.01421078
View 1 Train 0.69576163 0.67424643 0.71727484 0.02033194
View 1 CV 0.61750698 0.59387084 0.64114312 0.01355002
ROC Plot
ROC Plot
Sensitivity
1-specificity
Precision recall plot
PR Plot
Precision
Recall
F1 Score Plot
F1-Score Plot
F1-Score
Box Plot



Remember these plots pop up automatically. You don't have to do anything at all but run a line of code.


OHDSI Analysis
About
Prediction
Prediction Viewer
Back To Models Summary
Full Result Explorer
modelDesignId : 1
developmentDb : cdm_truven_mdcd_v2359
modelType : logistic
validationDb : cdm_truven_mdcd_v2359
Target : [OHDSI2022] New users of lisinopril with prior hypertension
outcome : [OHDSI2022] Angioedema events
Design Settings
Model
Threshold Dependant
Discrimination
Calibration
Net Benefit
Validation
Select net benefit type to view:
Type:
Test
Net Benefit Plot
Summary
Show 10 entries
Search:
pt
netBenefit
treatAll
1 0.053496927911474 -0.0000022648 -0.0485190
2 0.050365192285061 -0.0000042504 -0.0450614
3 0.048282474347483 -0.0000080988 -0.0427744
4 0.047243614990413 6 -0.0000079478 -0.0416374
5 0.046748673627001 -0.0000098256 -0.0410966
6 0.046589604989747 -0.0000117486 -0.0409229
7 0.045455083310582 -0.0000133570 -0.0396857
8 0.044892222244599 0.0000268876 -0.0390897
9 0.041970844824565 0.0000280287 -0.0359045
10 0.041821894925169 0.0000244084 -0.0355271
Showing 1 to 10 of 201 entries
Previous
1
2
3
4
5
21
Next

Example 2: Stroke Risk in Atrial Fibrillation

- Target condition: atrial fibrillation
- Outcome: ischemic stroke
- Time horizon: 1 year
- Motivation: improve risk stratification

Atrial fibrillation is a classic prediction problem in epidemiology. Patients with AF are at elevated risk of ischemic stroke.

Prediction Question

- Among patients newly diagnosed with atrial fibrillation, who will experience ischemic stroke within 1 year?

This sentence defines the prediction problem and drives all downstream design choices.

Defining the Target Population

- Newly diagnosed atrial fibrillation
- First qualifying event only
- Requires supporting evidence

The target population is defined using the first qualifying atrial fibrillation event with supporting clinical evidence.

Defining the Outcome

- Ischemic stroke
- Inpatient or emergency setting
- Events >180 days apart treated as independent

The outcome definition specifies what counts as ischemic stroke and how recurrent events are handled.

Time-at-Risk Definition

- Starts 1 day after cohort entry
- Ends 365 days later
- Represents 1-year risk window

The time-at-risk window defines when outcomes are counted.

Observation Time and Lookback

- Require prior observation time
- Enable baseline feature extraction
- Tradeoff between history and sample size

Requiring prior observation time ensures sufficient baseline data but reduces cohort size.

Handling Prior Outcomes

- Allow patients with prior stroke
- Prior outcomes are predictive
- Consistent with evidence logic

Patients with prior strokes are included because prior stroke strongly predicts future stroke risk.

Prediction Algorithms PatientLevelPrediction

Algorithm	What it does	Why you might use it
Regularized Logistic Regression (LASSO)	Linear model with automatic feature selection	Strong baseline; interpretable; performs well in many settings
Naive Bayes	Probabilistic classifier assuming feature independence	Fast, simple, and often surprisingly competitive
Decision Tree	Rule-based partitioning of predictors	Easy to interpret; useful for simple decision rules
Random Forest	Ensemble of decision trees (bagging)	Captures non-linearities and interactions; robust performance
Gradient Boosting Machines	Sequentially boosted decision trees	Often strong predictive performance in structured data
Neural Networks (MLP)	Non-linear models with hidden layers	Flexible function approximation when signal is complex
Deep Learning (separate package)	Multi-layer neural architectures	Explores latent representations; research-oriented

This table summarizes the main prediction algorithms available in OHDSI PatientLevelPrediction. All of these are well-established methods in statistics or machine learning. OHDSI does not require using any particular algorithm; instead, it provides a consistent framework to compare them fairly under the same study design.

Model Interpretability in OHDSI Prediction Studies

- Regression coefficients for linear models
- Feature importance for tree-based models
- Calibration plots for absolute risk
- Performance metrics reported side-by-side

Interpretability is treated as a first-class concern in OHDSI prediction studies. For linear models, coefficients can be directly inspected. For tree-based and ensemble models, feature importance summaries are available. Calibration plots are routinely used to assess whether predicted risks correspond to observed outcomes. These tools allow users to balance predictive performance with interpretability.

Problem Definition	
Target Cohort (T)	'Patients who are newly diagnosed with Atrial Fibrillation' defined as the first condition record of cardiac arrhythmia, which is followed by another cardiac arrhythmia condition record, at least two drug records for a drug used to treat arrhythmia, or a procedure to treat arrhythmia.
Outcome Cohort (O)	'Ischemic stroke events' defined as ischemic stroke condition records during an inpatient or ER visit; successive records with > 180 day gap are considered independent episodes.
Time-at-risk (TAR)	1 day till 365 days from cohort start
Population Definition	
Washout Period	1095
Enter the target cohort multiple times?	No
Allow prior outcomes?	Yes
Start of time-at-risk	1 day
End of time-at-risk	365 days
Require a minimum amount of time-at-risk?	Yes (364 days)

So in HADES, like any other good science, you still need to make a protocol and do everything else you would need to do for an analysis outside of OHDSI. What I like about OHDSI is that it lays all of this stuff out nicely for me so I don't need to reinvent the wheel.

According to the best practices we need to make a protocol that completely specifies how we plan to execute our study. This protocol will be assessed by the governance boards of the participating data sources in your network study. For this a template could be used but we prefer to automate this process as much as possible by adding functionality to automatically generate study protocol from a study specification.

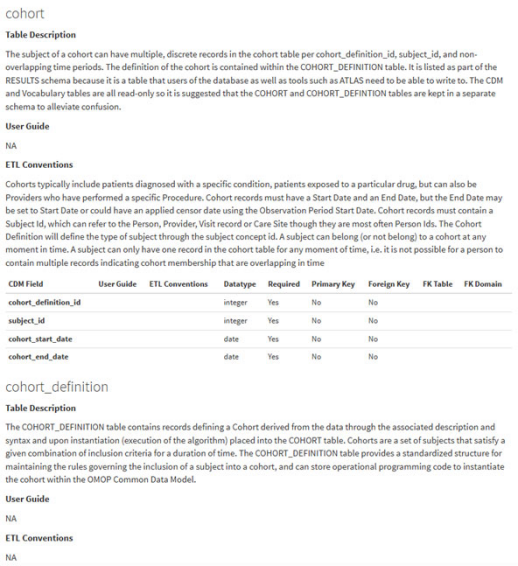
Model Development	
Algorithm	Regularized Logistic Regression
Hyper-parameters	variance = 0.01 (Default)
Covariates	Gender, Age, Conditions (ever before, <365), Drugs Groups (ever before, <365), and Visit Count
Data split	75% train, 25% test. Randomly assigned by person

So you write all of this up ahead of time just like you would with any other study. And there's even a package, maybe Chenyre has used it, that helps you to auto-generate your protocol. As I said, they have thought of everything.

Study Implementation
Setting up the analysis using Atlas + R

Now that we have completely designed our study we have to implement the study. We have to generate the target and outcome cohorts and we need to develop the R code to run against our CDM that will execute the full study.

Cohort instantiation



cohort

Table Description

The subject of a cohort can have multiple, discrete records in the cohort table per cohort_definition_id, subject_id, and non-overlapping time periods. The definition of the cohort is contained within the COHORT_DEFINITION table. It is listed as part of the RESULTS schema because it is a table that users of the database as well as tools such as ATLAS need to be able to write to. The CDM and Vocabulary tables are all read-only so it is suggested that the COHORT and COHORT_DEFINTION tables are kept in a separate schema to alleviate confusion.

User Guide

NA

ETL Conventions

Cohorts typically include patients diagnosed with a specific condition, patients exposed to a particular drug, but can also be Providers who have performed a specific Procedure. Cohort records must have a Start Date and an End Date, but the End Date may be set to Start Date or could have an applied censor date using the Observation Period Start Date. Cohort records must contain a Subject Id, which can refer to the Person, Provider, Visit record or Care Site though they are most often Person Ids. The Cohort Definition will define the type of subject through the subject concept id. A subject can belong (or not belong) to a cohort at any moment in time. A subject can only have one record in the cohort table for any moment of time, i.e. it is not possible for a person to contain multiple records indicating cohort membership that are overlapping in time

CDM Field	User Guide	ETL Conventions	Datatype	Required	Primary Key	Foreign Key	FK Table	FK Domain
cohort_definition_id			integer	Yes	No	No		
subject_id			integer	Yes	No	No		
cohort_start_date			date	Yes	No	No		
cohort_end_date			date	Yes	No	No		

cohort_definition

Table Description

The COHORT_DEFINITION table contains records defining a Cohort derived from the data through the associated description and syntax and upon instantiation (execution of the algorithm) placed into the COHORT table. Cohorts are a set of subjects that satisfy a given combination of inclusion criteria for a duration of time. The COHORT_DEFINITION table provides a standardized structure for maintaining the rules governing the inclusion of a subject into a cohort, and can store operational programming code to instantiate the cohort within the OMOP Common Data Model.

User Guide

NA

ETL Conventions

NA

For our study we need to know when a person enters the target and outcome cohorts. This is stored in a table on the server that contains the cohort start date and cohort end date for all subjects for a specific cohort definition. This cohort table has a very simple structure as shown below:

cohort_definition_id, a unique identifier for distinguishing between different types of cohorts, e.g. cohorts of interest and outcome cohorts.

subject_id, a unique identifier corresponding to the person_id in the CDM.

cohort_start_date, the date the subject enters the cohort.

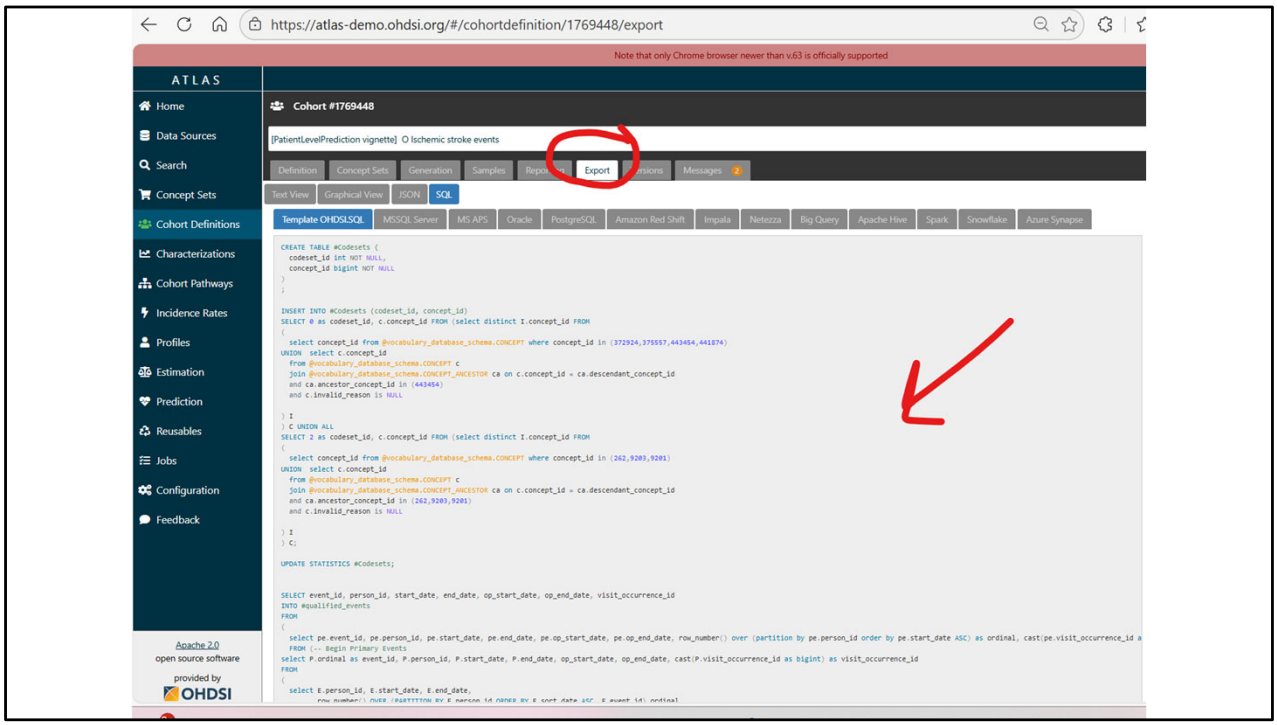
cohort_end_date, the date the subject leaves the cohort.

How do we fill this table according to our cohort definitions? There are two options for this:

use the interactive cohort builder tool in ATLAS which can be used to create cohorts based on inclusion criteria and will automatically populate this cohort table.

write your own custom SQL statements to fill the cohort table.

For today we will use the Atlas cohort builder since you are already so familiar with Atlas



Again, in the interest of showing you how all of these tools work together, recall that you can "export" your cohort definition SQL code from Atlas. This is the basic structure that you would use if "starting from scratch."

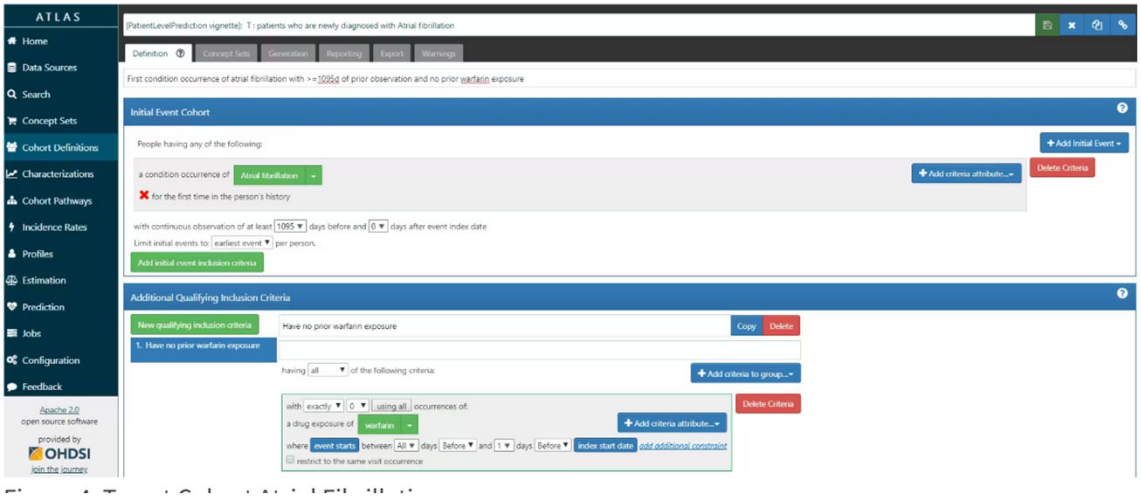
Why would you want to do this outside of Atlas? Well for one thing, some people don't have Atlas but they do have an OMOP CDM and R. I have worked with dozens of hospitals and nonprofits to get them started with OMOP. Some don't have the infrastructure to set up Atlas. So there is code available to make your own cohort definitions. It mimics the "under the hood" code used by Atlas.

```
library(FeatureExtraction)
covariateSettings <- createCovariateSettings(
  useDemographicsGender = TRUE,
  useDemographicsAge = TRUE,
  useConditionGroupEraLongTerm = TRUE,
  useConditionGroupEraAnyTimePrior = TRUE,
  useDrugGroupEraLongTerm = TRUE,
  useDrugGroupEraAnyTimePrior = TRUE,
  useVisitConceptCountLongTerm = TRUE,
  longTermStartDays = -365,
  endDays = -1
)
```

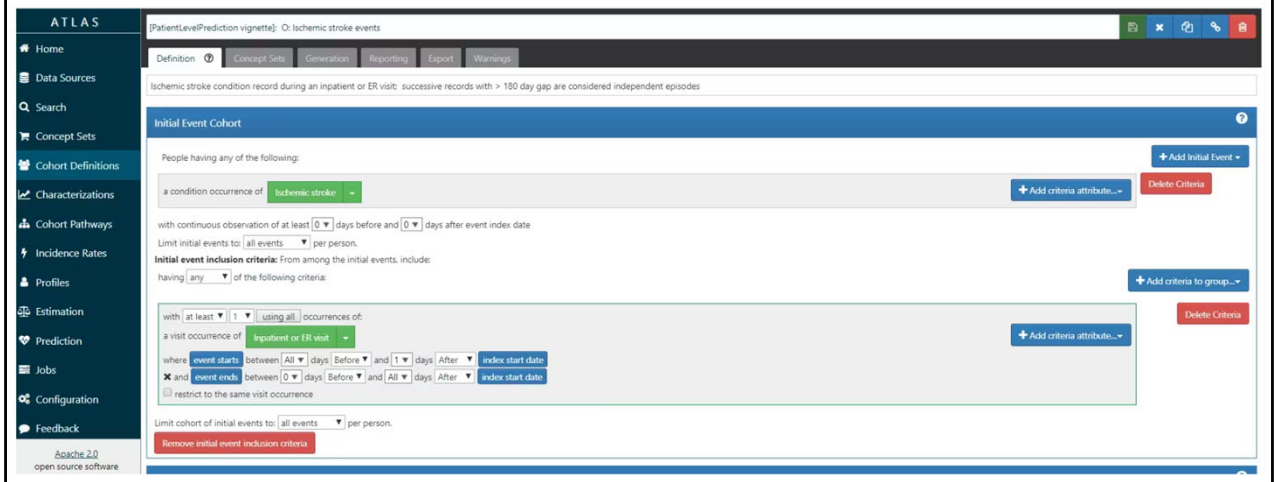
Now, back in R, we can tell PatientLevelPrediction to extract all necessary data for our analysis. Just like we did in the first example. This is done using the FeatureExtraction package. In short the FeatureExtraction package allows you to specify which features (covariates) need to be extracted, e.g. all conditions and drug exposures. It also supports the creation of custom covariates. For more detailed information on the FeatureExtraction package see its vignettes. Note that even some of this design work I am showing you in R can also be done in Atlas. I will show you that in a minute.

```
databaseDetails <- createDatabaseDetails(
  connectionDetails    = connectionDetails,
  cdmDatabaseSchema    = cdmDatabaseSchema,
  cohortDatabaseSchema  = resultsDatabaseSchema,
  cohortTable         = "cohort",
  outcomeDatabaseSchema = resultsDatabaseSchema,
  outcomeTable        = "cohort"
)

restrictPlpDataSettings <- createRestrictPlpDataSettings(
  sampleSize = 10000
)

plpData <- getPlpData(
  databaseDetails       = databaseDetails,
  covariateSettings      = covariateSettings,
  cohortId             = 12345,      # AFib cohortDefinitionId (from Atlas)
  outcomeIds           = c(56789),    # Stroke cohortDefinitionId(s) (from Atlas)
  restrictPlpDataSettings  = restrictPlpDataSettings
)
```

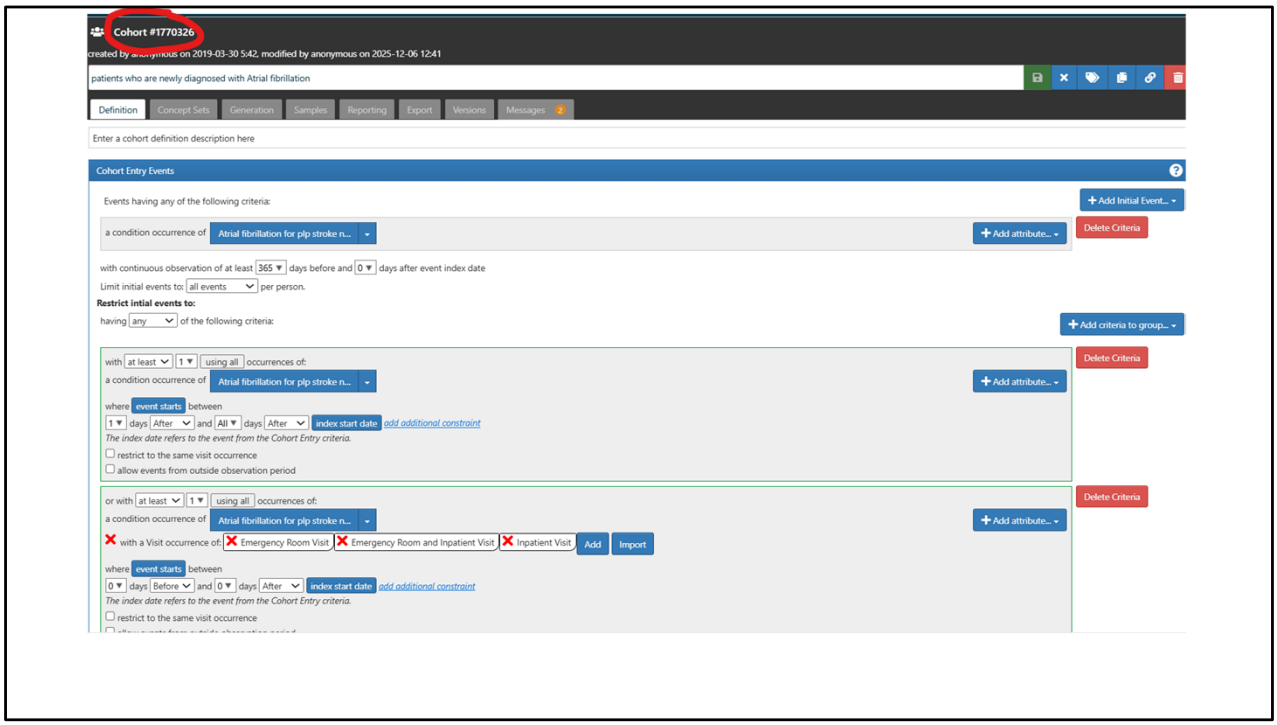
The final step for extracting the data is to run the getPlpData() function and input the connection details, the database schema where the cohorts are stored, the cohort definition ids for the cohort and outcome, and the washoutPeriod which is the minimum number of days prior to cohort index date that the person must have been observed to be included into the data, and finally input the previously constructed covariateSettings. In this example, in red, is where you would put your Atlas cohort definition #s.



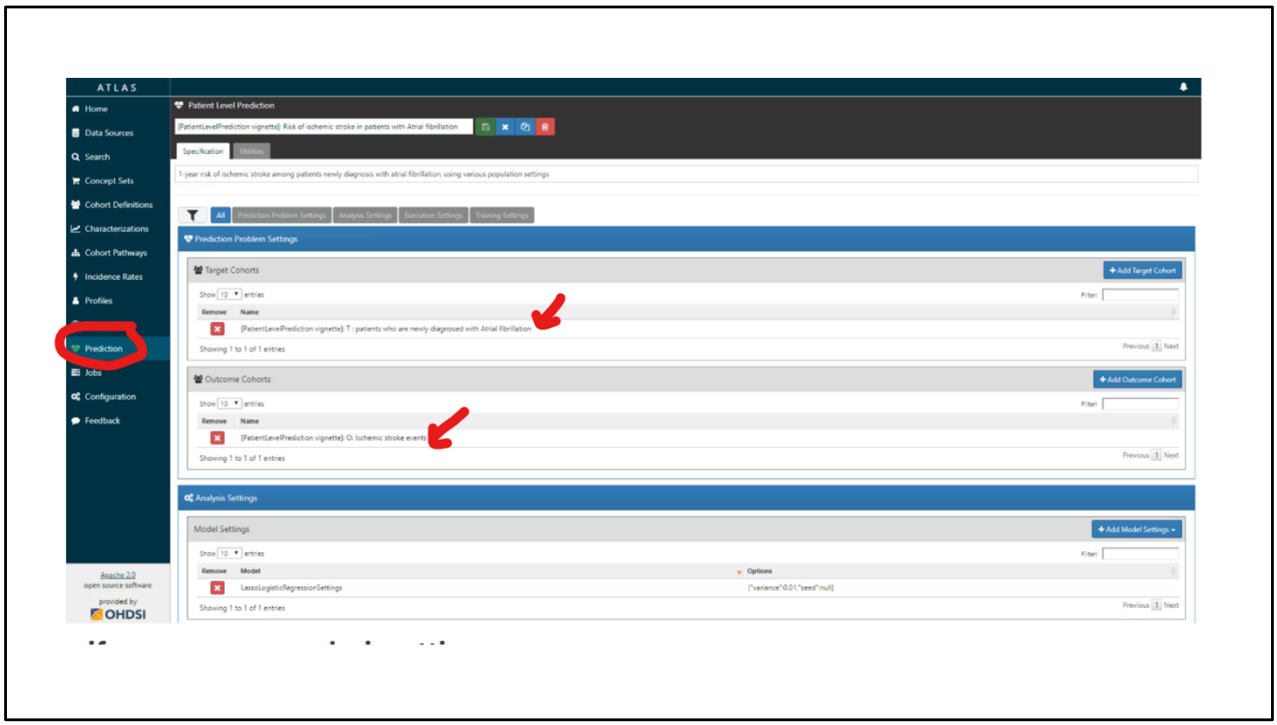
Let me show you how this looks if you are using Atlas for some of these steps before moving over to HADES for the execution alone.
ATLAS allows you to define cohorts interactively by specifying cohort entry and cohort exit criteria. As a reminder, cohort entry criteria involve selecting one or more initial events, which determine the start date for cohort entry, and optionally specifying additional inclusion criteria which filter to the qualifying events. Cohort exit criteria are applied to each cohort entry record to determine the end date when the person's episode no longer qualifies for the cohort. For the outcome cohort the end date is less relevant. As an example, this shows how we created the Atrial Fibrillation cohort and the next shows how we created the stroke cohort in ATLAS. In a minute I will show you how all of this looks in SEARCH.



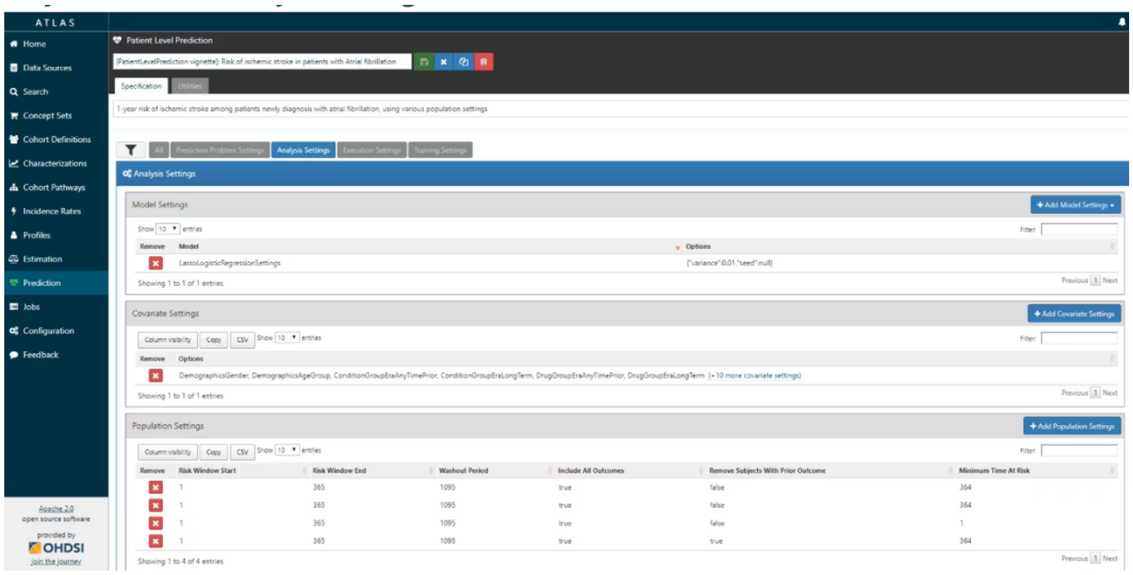
Here is the stroke outcome cohort. You know how to do this.



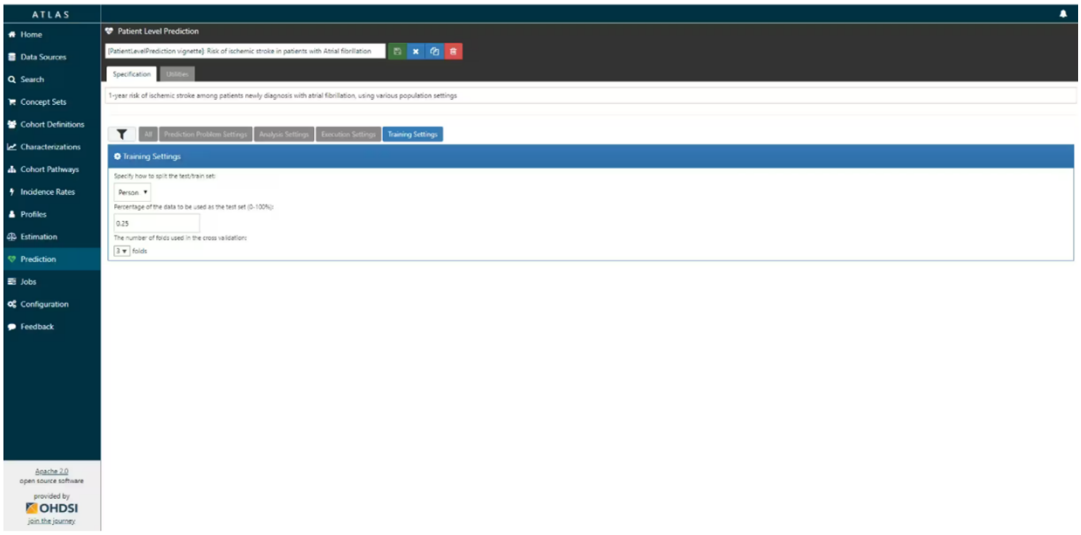
Remember: your Atlas connects to the same CDM as your R. When you generate a cohort in Atlas it goes into your CDM.



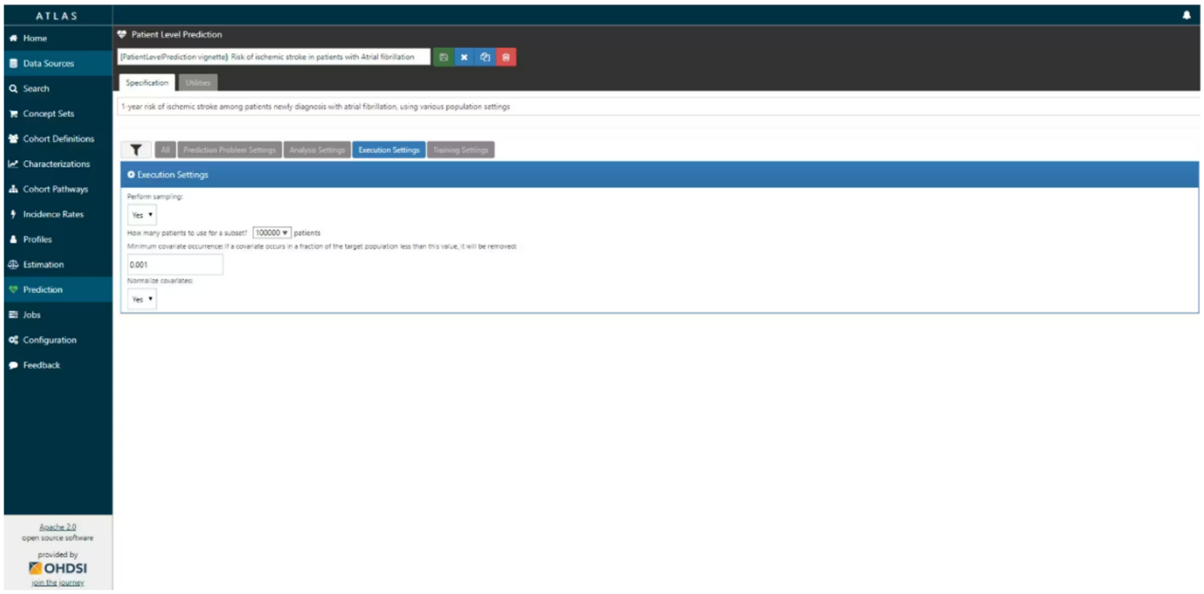
The setting specification script we created manually before can also be automatically created using a powerful feature in ATLAS. By creating a new prediction study (left menu) you can select the Target and Outcome as created in ATLAS, set all the study parameters, and then you can download a R package that you can use to execute your study. What is really powerful is that you can add multiple Ts, Os, covariate settings etc. The package will then run all the combinations of automatically as separate analyses. The screenshots here explain this process. Forst, create a new prediction study and select your targer and outcome cohorts. This is similar to how we plugged in cohorts of interest when we did the Treatment Pathway analysis last week.



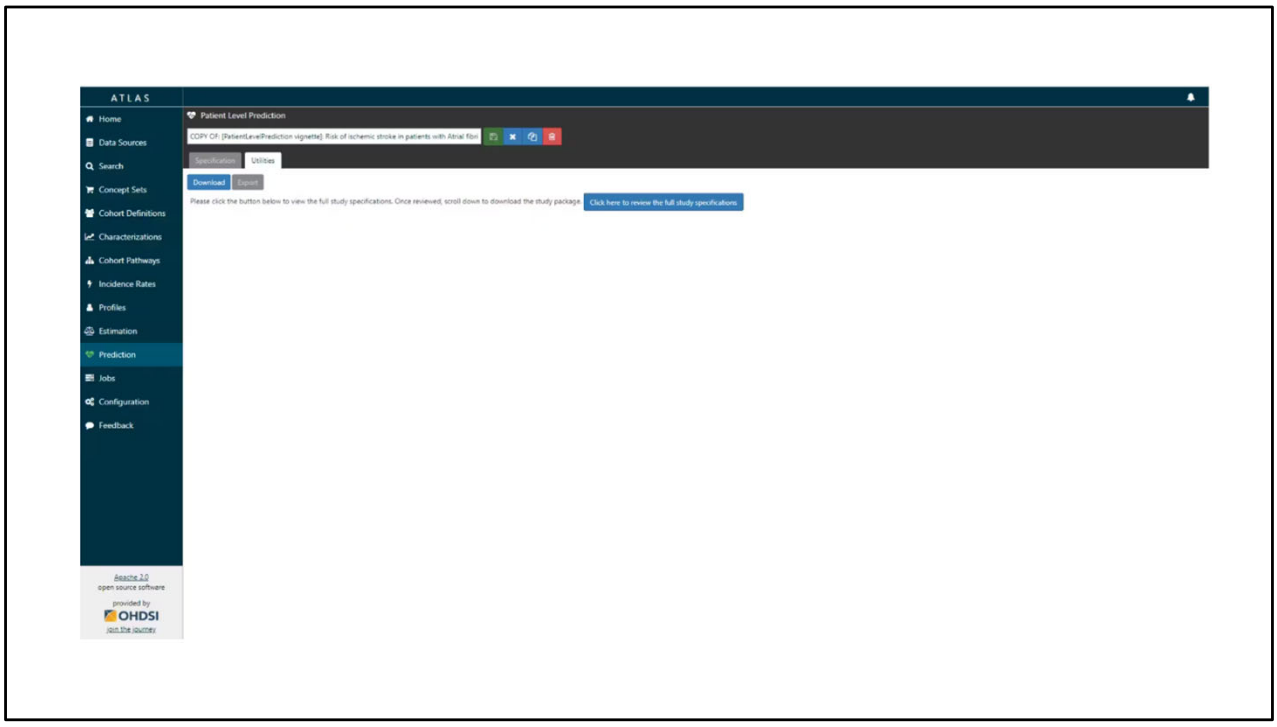
Next, specify one or more analysis settings. See demographics? We wrote that out in code in the earlier example. But if you want to save the step you can do some of these steps right here within Atlas.



Specify the analysis settings

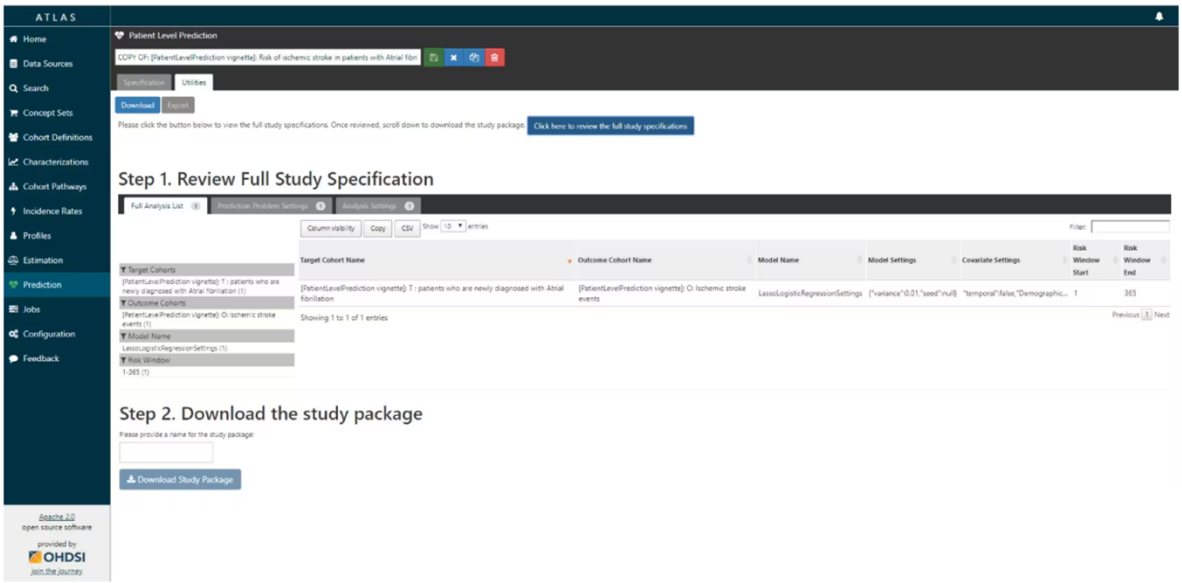


Specify the execution settings



ATLAS can build a R package for you that will execute the full study against you CDM. Below the steps are explained how to do this in ATLAS.

Under utilities you can find download. Click on the button to review the full study specification

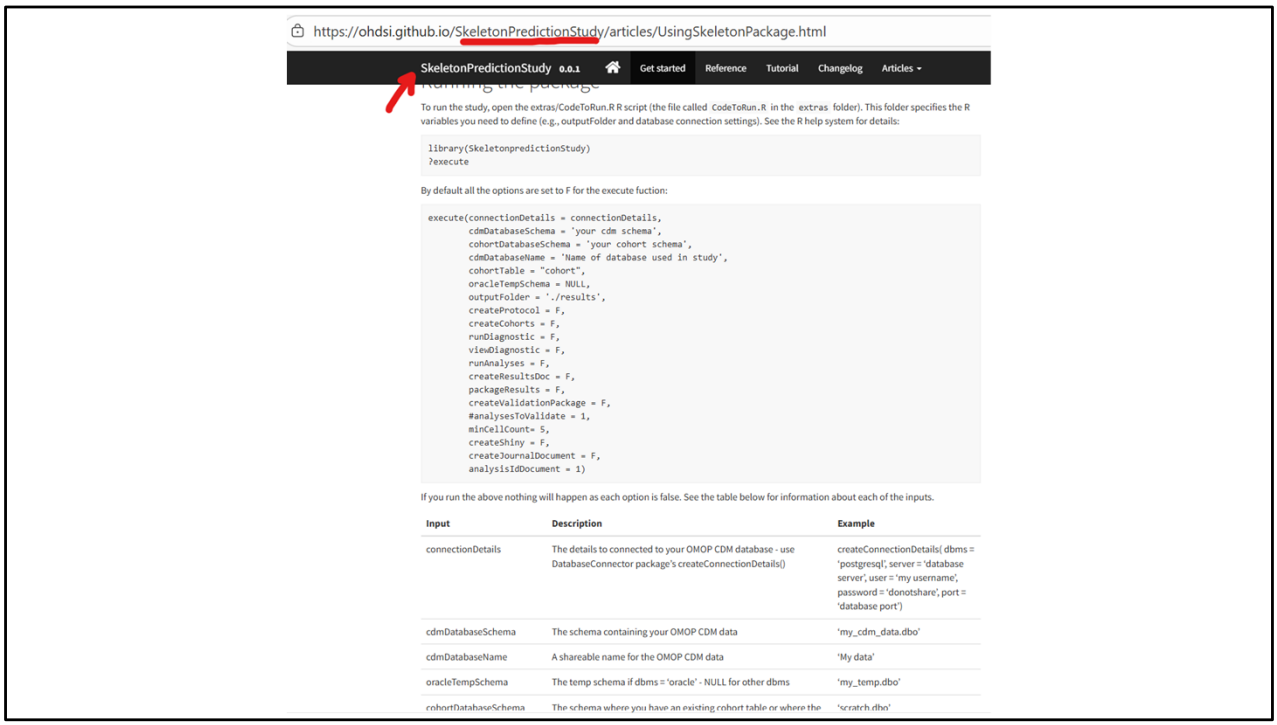


R package download functionality in ATLAS

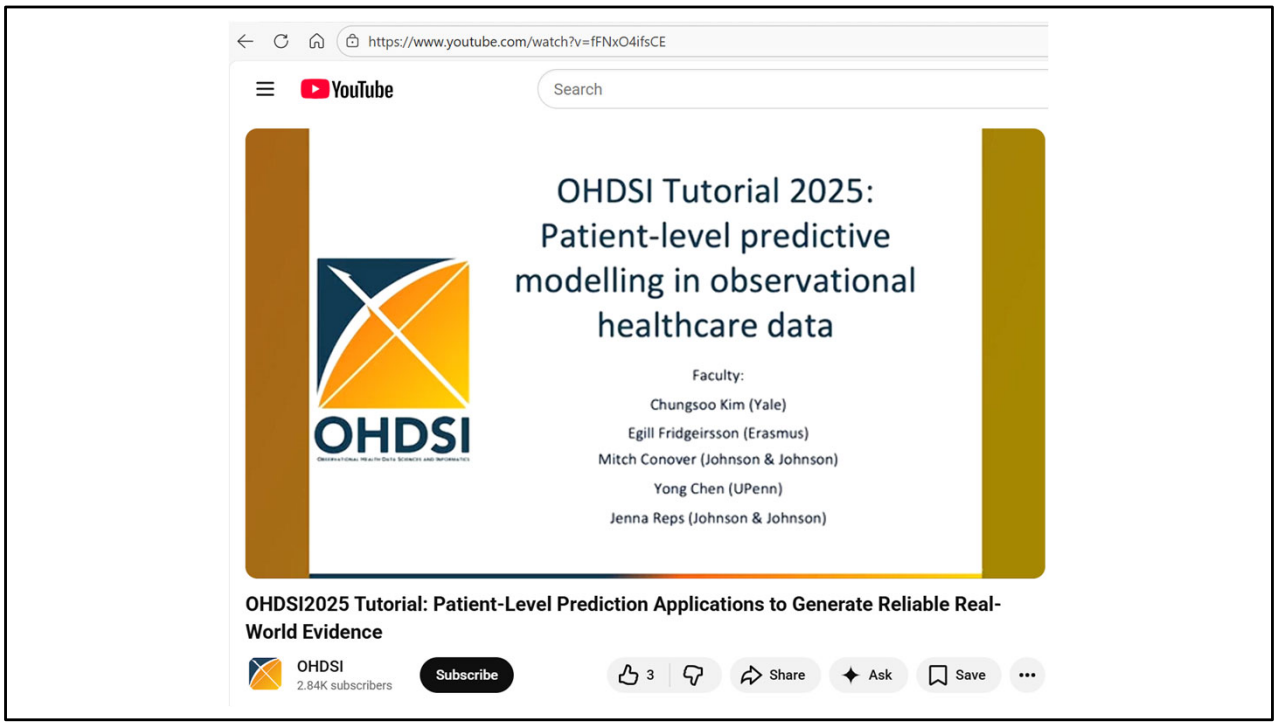
You now have to review that you indeed want to run all these analyses.

If you agree, you give the package a name, and download the package as a zipfile.

By opening the R package in R studio and building the package you can run the study using the execute function. Theres is also an example CodeToRun.R script available in the extras folder of the package with extra instructions. They have truly thought of everything.


https://ohdsi.github.io/SkeletonPredictionStudy/articles/UsingSkeletonPackage.html
SkeletonPredictionStudy 0.0.1
Get started
Reference
Tutorial
Changelog
Articles
To run the study, open the extras/CodeToRun.R R script (the file called CodeToRun.R in the extras folder). This folder specifies the R variables you need to define (e.g., outputFolder and database connection settings). See the R help system for details:
library(SkeletonpredictionStudy)
?execute
By default all the options are set to F for the execute fuction:
execute(connectionDetails = connectionDetails,
cdmDatabaseSchema = 'your cdm schema',
cohortDatabaseSchema = 'your cohort schema',
cdmDatabaseName = 'Name of database used in study',
cohortTable = "cohort",
oracleTempSchema = NULL,
outputFolder = './results',
createProtocol = F,
createCohorts = F,
runDiagnostic = F,
viewDiagnostic = F,
runAnalyses = F,
createResultsDoc = F,
packageResults = F,
createValidationPackage = F,
#analysesToValidate = 1,
minCellCount= 5,
createShiny = F,
createJournalDocument = F,
analysisIdDocument = 1)
If you run the above nothing will happen as each option is false. See the table below for information about each of the inputs.
Input
Description
Example
connectionDetails
The details to connected to your OMOP CDM database - use DatabaseConnector package's createConnectionDetails()
createConnectionDetails(dbms = 'postgresql', server = 'database server', user = 'my username', password = 'donotshare', port = 'database port')
cdmDatabaseSchema
The schema containing your OMOP CDM data
'my_cdm_data.dbo'
cdmDatabaseName
A shareable name for the OMOP CDM data
'My data'
oracleTempSchema
The temp schema if dbms = 'oracle' - NULL for other dbms
'my_temp.dbo'
cohortDatabaseSchema
The schema where you have an existing cohort table or where the
'scratch.dbo'

For more information on additional features and functionality, you would start here. This page also includes the citation – important to cite the package when you use OHDSI, just like you would cite STATA, SPSS, SAS, etc.


https://www.youtube.com/watch?v=fFNxO4ifsCE
YouTube
Search
OHDSI Tutorial 2025:
Patient-level predictive
modelling in observational
healthcare data
Faculty:
Chungsoo Kim (Yale)
Egill Fridgeirsson (Erasmus)
Mitch Conover (Johnson & Johnson)
Yong Chen (UPenn)
Jenna Reps (Johnson & Johnson)
OHDSI
OHDSI2025 Tutorial: Patient-Level Prediction Applications to Generate Reliable Real-World Evidence
OHDSI
2.84K subscribers
Subscribe
3
Share
Ask
Save

Staying connected

THANK YOU!

Extra slides

```
plotPlp(lrResults, dirPath = file.path(tempdir(), "plots"))
```

To generate and save all the evaluation plots to a folder run the following code:
plotPlp(lrResults, dirPath = file.path(tempdir(), "plots"))
The plots are described in more detail in the online documentation – that's probably out of scope for this training.

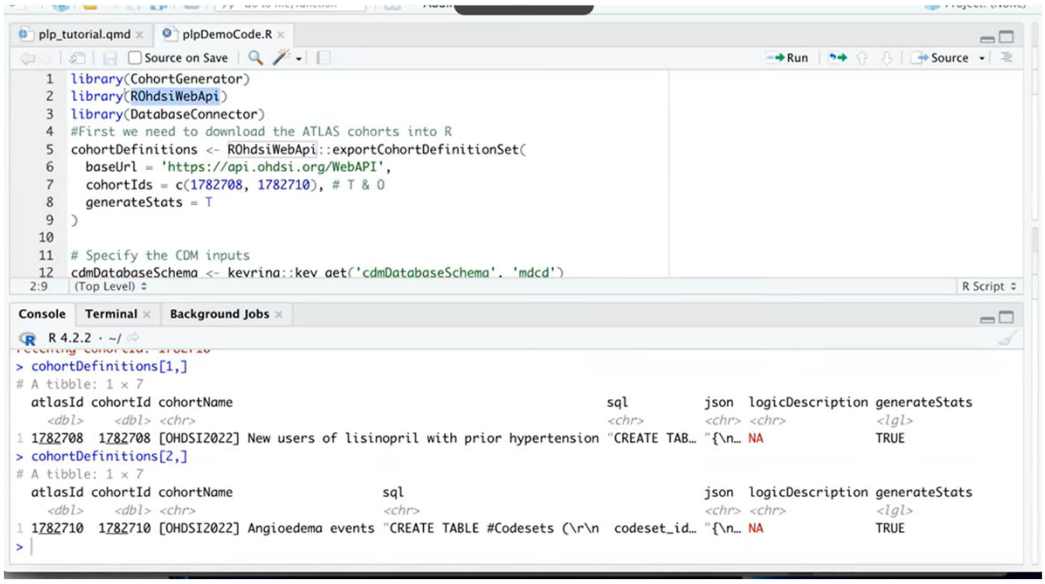
External validation

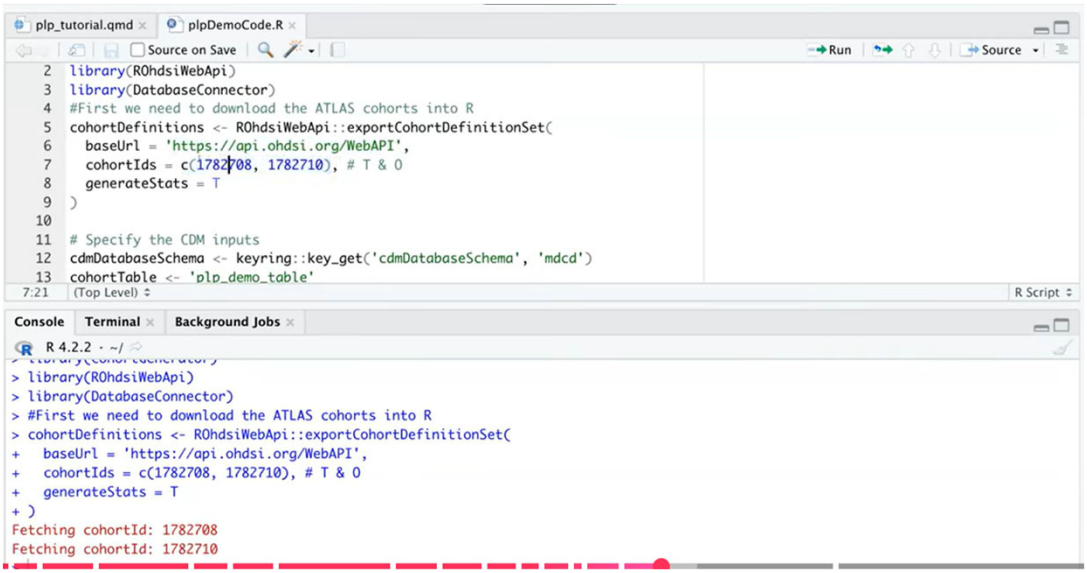
```
# load the trained model
plpModel <- loadPlpModel(file.path(tempdir(), "model"),
"model")

# add details of new database
validationDatabaseDetails <- createDatabaseDetails()

# to externally validate the model and perform recalibration
run:
externalValidateDbPlp(
 plpModel = plpModel,
 validationDatabaseDetails = validationDatabaseDetails,
 validationRestrictPlpDataSettings =
plpModel$settings$plpDataSettings,
 settings = createValidationSettings(
  recalibrate = "weakRecalibration"
 ),
 outputFolder = file.path(tempdir(), "validation")
)
```

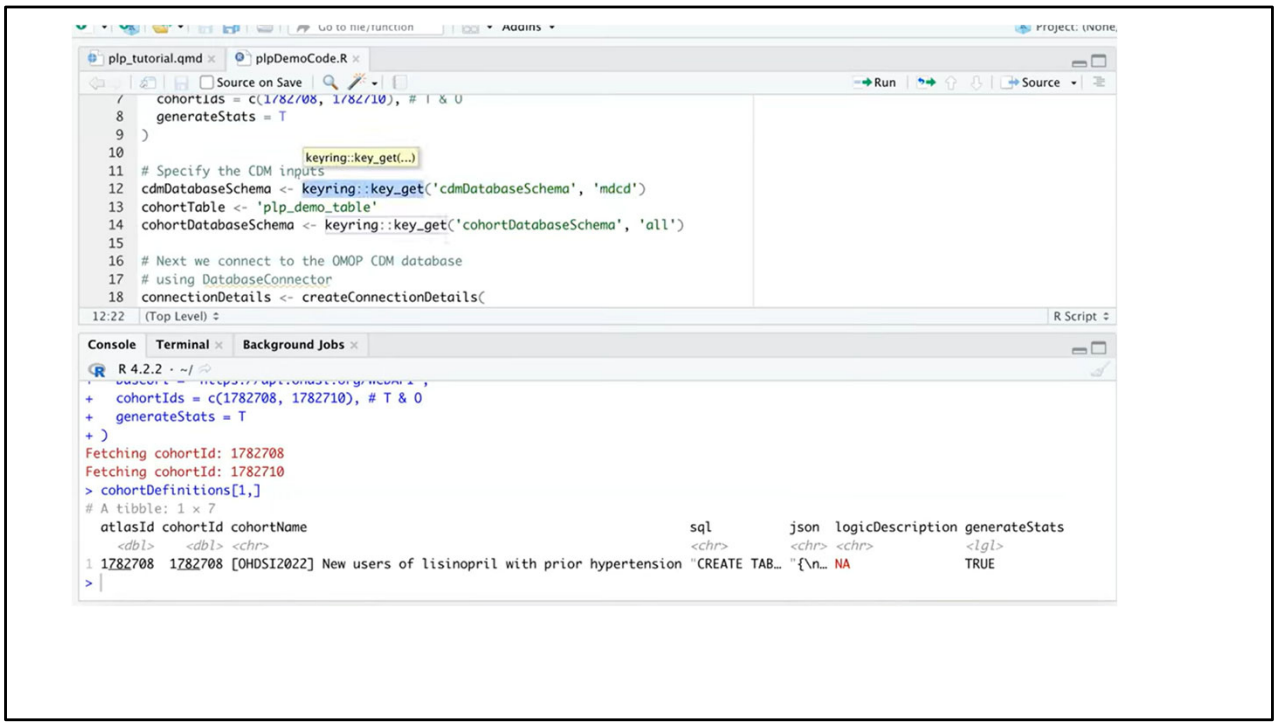
External validation

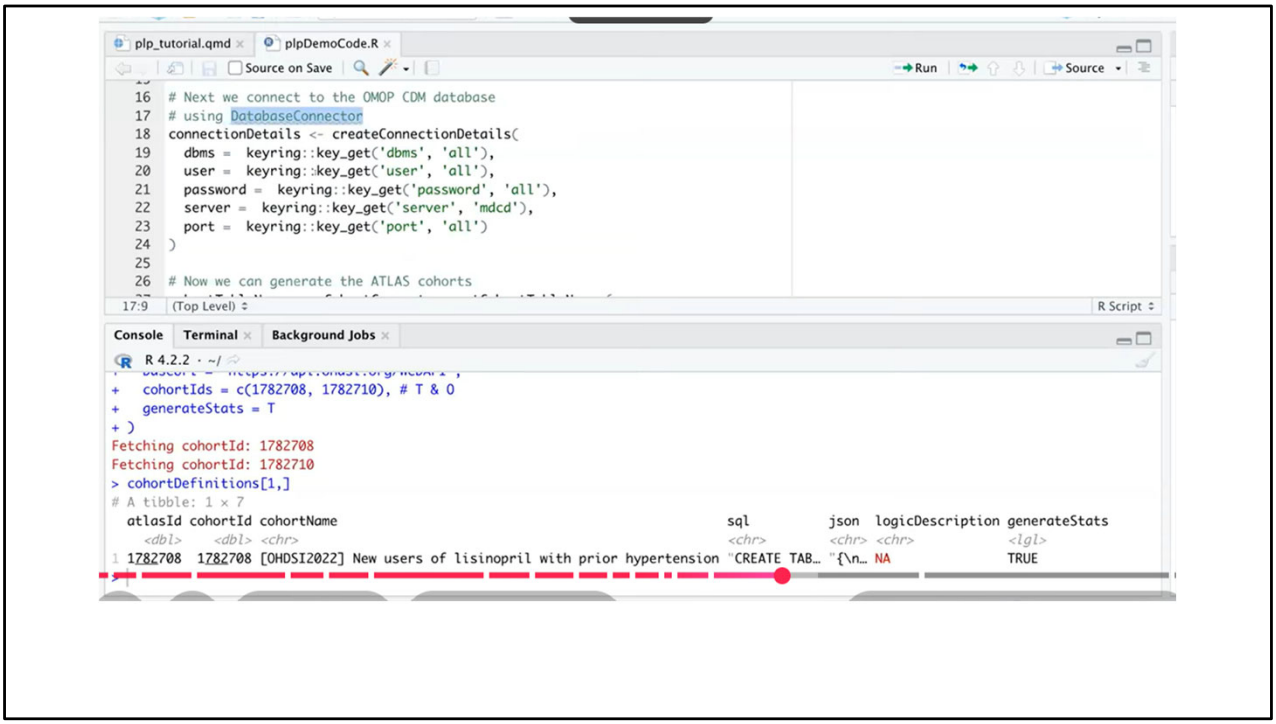
We recommend to always perform external validation, i.e. apply the final model on as much new datasets as feasible and evaluate its performance.

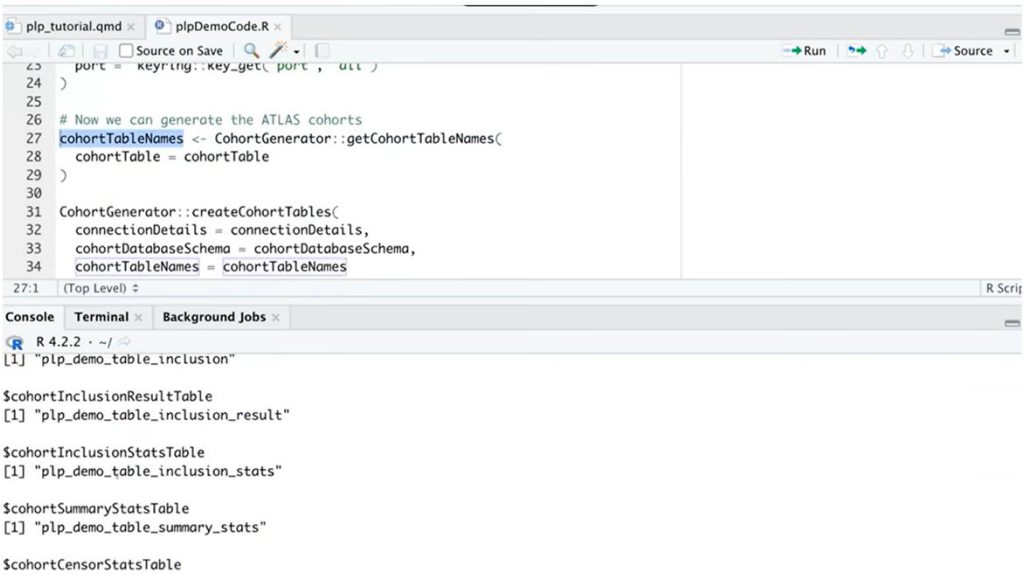
This will extract the new plpData from the specified schemas and cohort tables. It will then apply the same population settings and the trained plp model. Finally, it will evaluate the performance and return the standard output as validation$performanceEvaluation and it will also return the prediction on the population as validation$prediction. They can be inserted into the shiny app for viewing the model and validation by running: viewPlp(runPlp=plpResult, validatePlp=validation).

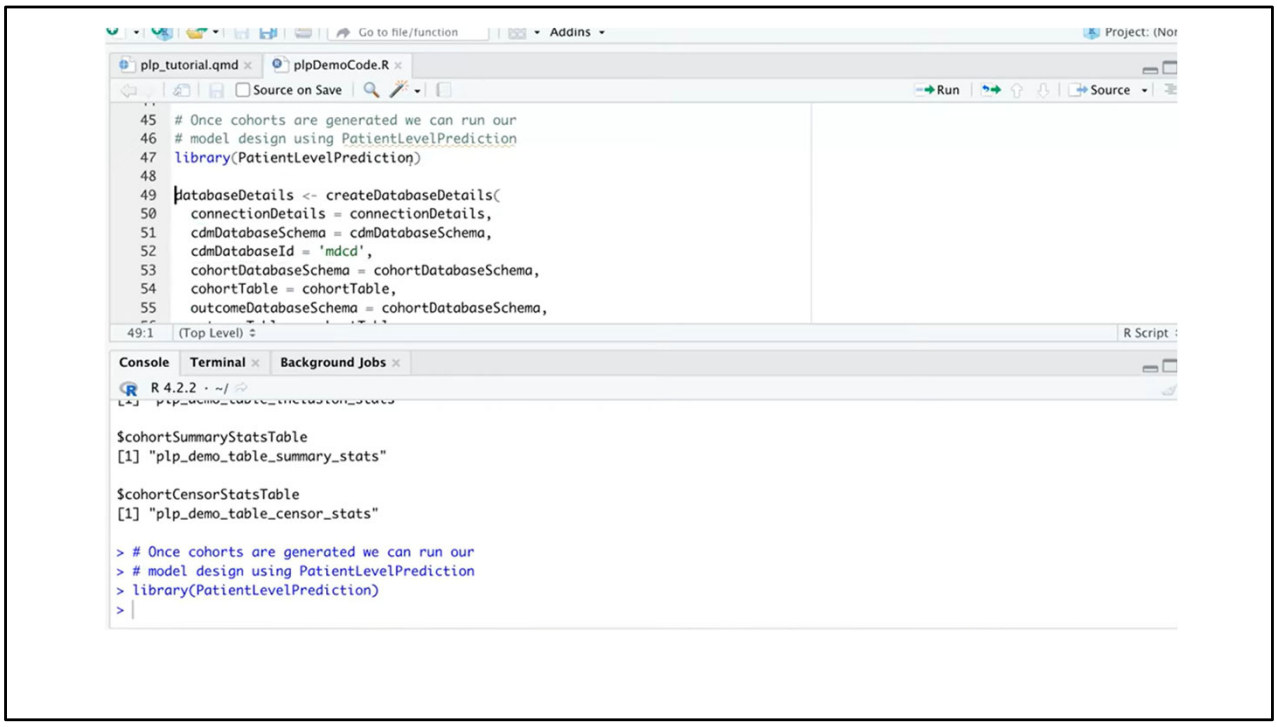

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
1 library(CohortGenerator)
2 library(ROhdsiWebApi)
3 library(DatabaseConnector)
4 #First we need to download the ATLAS cohorts into R
5 cohortDefinitions <- ROhdsiWebApi::exportCohortDefinitionSet(
6 baseUrl = 'https://api.ohdsi.org/WebAPI',
7 cohortIds = c(1782708, 1782710), # T & O
8 generateStats = T
9)
10
11 # Specify the CDM inputs
12 cdmDatabaseSchema <- keyring::key_get('cdmDatabaseSchema', 'mdcd')
2:9
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
> cohortDefinitions[1,]
A tibble: 1 × 7
atlasId cohortId cohortName sql json logicDescription generateStats
<dbl> <dbl> <chr> <chr> <chr> <chr> <lgl>
1 1782708 1782708 [OHDSI2022] New users of lisinopril with prior hypertension "CREATE TAB… "{\n… NA TRUE
> cohortDefinitions[2,]
A tibble: 1 × 7
atlasId cohortId cohortName sql json logicDescription generateStats
<dbl> <dbl> <chr> <chr> <chr> <chr> <lgl>
1 1782710 1782710 [OHDSI2022] Angioedema events "CREATE TABLE #Codesets (\r\n codeset_id… "{\n… NA TRUE
>

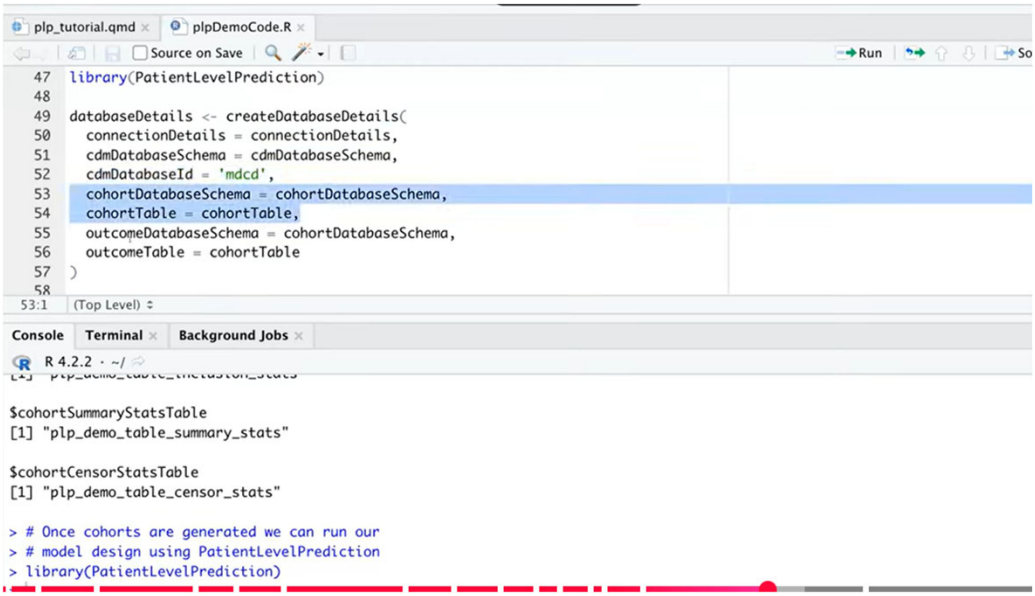

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
2 library(ROhdsiWebApi)
3 library(DatabaseConnector)
4 #First we need to download the ATLAS cohorts into R
5 cohortDefinitions <- ROhdsiWebApi::exportCohortDefinitionSet(
6 baseUrl = 'https://api.ohdsi.org/WebAPI',
7 cohortIds = c(1782708, 1782710), # T & O
8 generateStats = T
9)
10
11 # Specify the CDM inputs
12 cdmDatabaseSchema <- keyring::key_get('cdmDatabaseSchema', 'mdcd')
13 cohortTable <- 'plp_demo_table'
7:21 (Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
> library(ROhdsiWebApi)
> library(DatabaseConnector)
> #First we need to download the ATLAS cohorts into R
> cohortDefinitions <- ROhdsiWebApi::exportCohortDefinitionSet(
+ baseUrl = 'https://api.ohdsi.org/WebAPI',
+ cohortIds = c(1782708, 1782710), # T & O
+ generateStats = T
+)
Fetching cohortId: 1782708
Fetching cohortId: 1782710

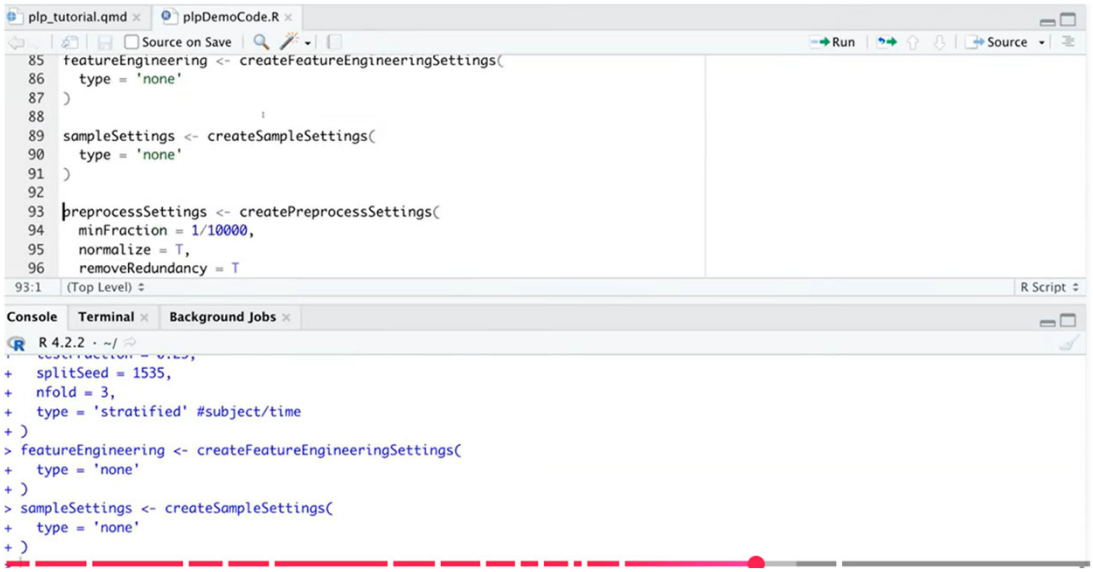

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
cohortIds = c(1782708, 1782710), # T & O
generateStats = T
)
keyring::key_get(...)
# Specify the CDM inputs
cdmDatabaseSchema <- keyring::key_get('cdmDatabaseSchema', 'mdcd')
cohortTable <- 'plp_demo_table'
cohortDatabaseSchema <- keyring::key_get('cohortDatabaseSchema', 'all')
# Next we connect to the OMOP CDM database
# using DatabaseConnector
connectionDetails <- createConnectionDetails(
12:22
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
+ cohortIds = c(1782708, 1782710), # T & O
+ generateStats = T
+)
Fetching cohortId: 1782708
Fetching cohortId: 1782710
> cohortDefinitions[1,]
A tibble: 1 × 7
atlasId cohortId cohortName sql json logicDescription generateStats
<dbl> <dbl> <chr> <chr> <chr> <chr> <lgl>
1 1782708 1782708 [OHDSI2022] New users of lisinopril with prior hypertension "CREATE TAB… "{\n… NA TRUE
>

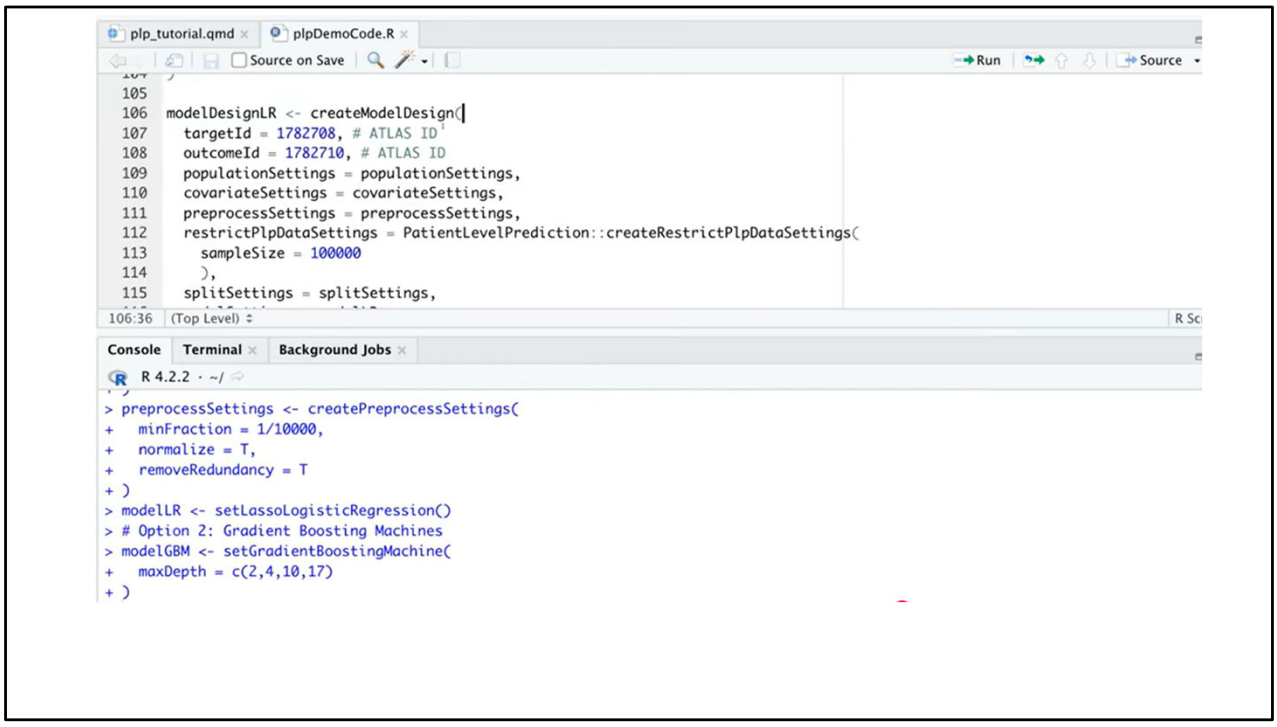

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
16 # Next we connect to the OMOP CDM database
17 # using DatabaseConnector
18 connectionDetails <- createConnectionDetails(
19 dbms = keyring::key_get('dbms', 'all'),
20 user = keyring::key_get('user', 'all'),
21 password = keyring::key_get('password', 'all'),
22 server = keyring::key_get('server', 'mdcd'),
23 port = keyring::key_get('port', 'all')
24)
25
26 # Now we can generate the ATLAS cohorts
17:9
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
+ cohortIds = c(1782708, 1782710), # T & O
+ generateStats = T
+)
Fetching cohortId: 1782708
Fetching cohortId: 1782710
> cohortDefinitions[1,]
A tibble: 1 × 7
atlasId cohortId cohortName sql json logicDescription generateStats
<dbl> <dbl> <chr> <chr> <chr> <chr> <lgl>
1 1782708 1782708 [OHDSI2022] New users of lisinopril with prior hypertension "CREATE TAB… "{\n… NA TRUE

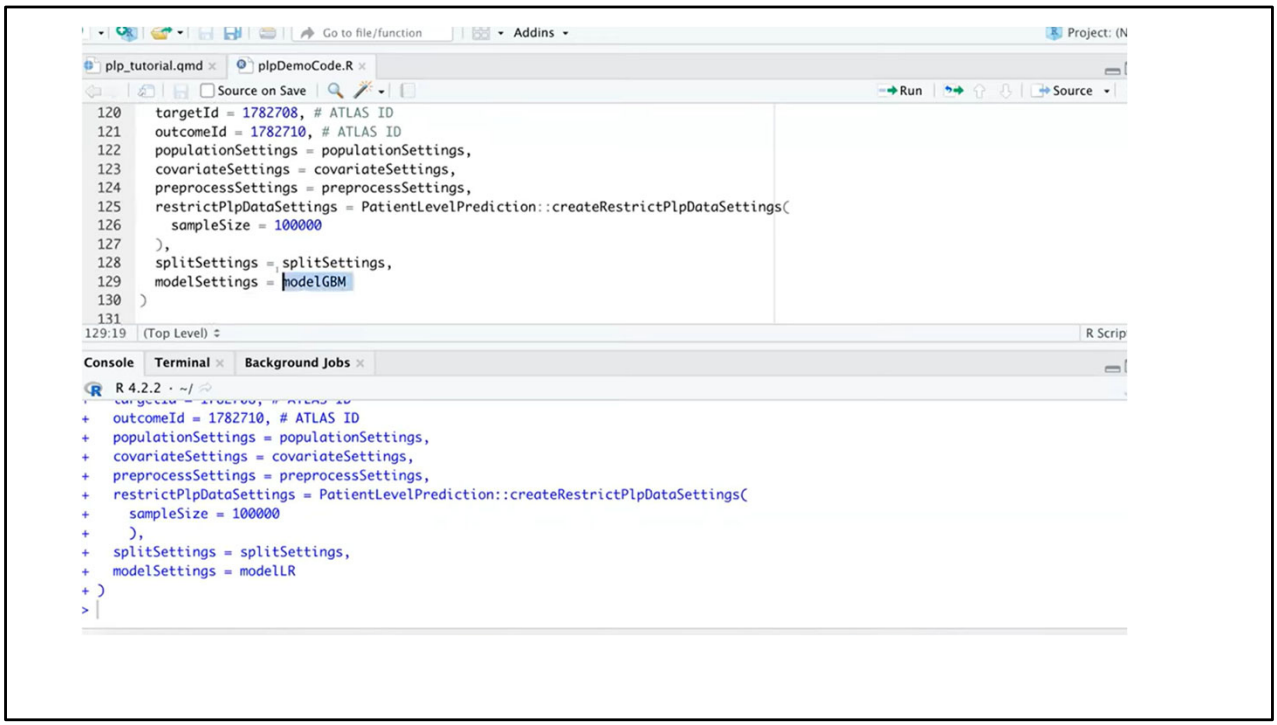

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
port = keyring::key_get("port", "all")
)
# Now we can generate the ATLAS cohorts
cohortTableNames <- CohortGenerator::getCohortTableNames(
cohortTable = cohortTable
)
CohortGenerator::createCohortTables(
connectionDetails = connectionDetails,
cohortDatabaseSchema = cohortDatabaseSchema,
cohortTableNames = cohortTableNames
27:1
(Top Level)
Console
Terminal
Background Jobs
R 4.2.2 · ~/
[1] "plp_demo_table_inclusion"
$cohortInclusionResultTable
[1] "plp_demo_table_inclusion_result"
$cohortInclusionStatsTable
[1] "plp_demo_table_inclusion_stats"
$cohortSummaryStatsTable
[1] "plp_demo_table_summary_stats"
$cohortCensorStatsTable

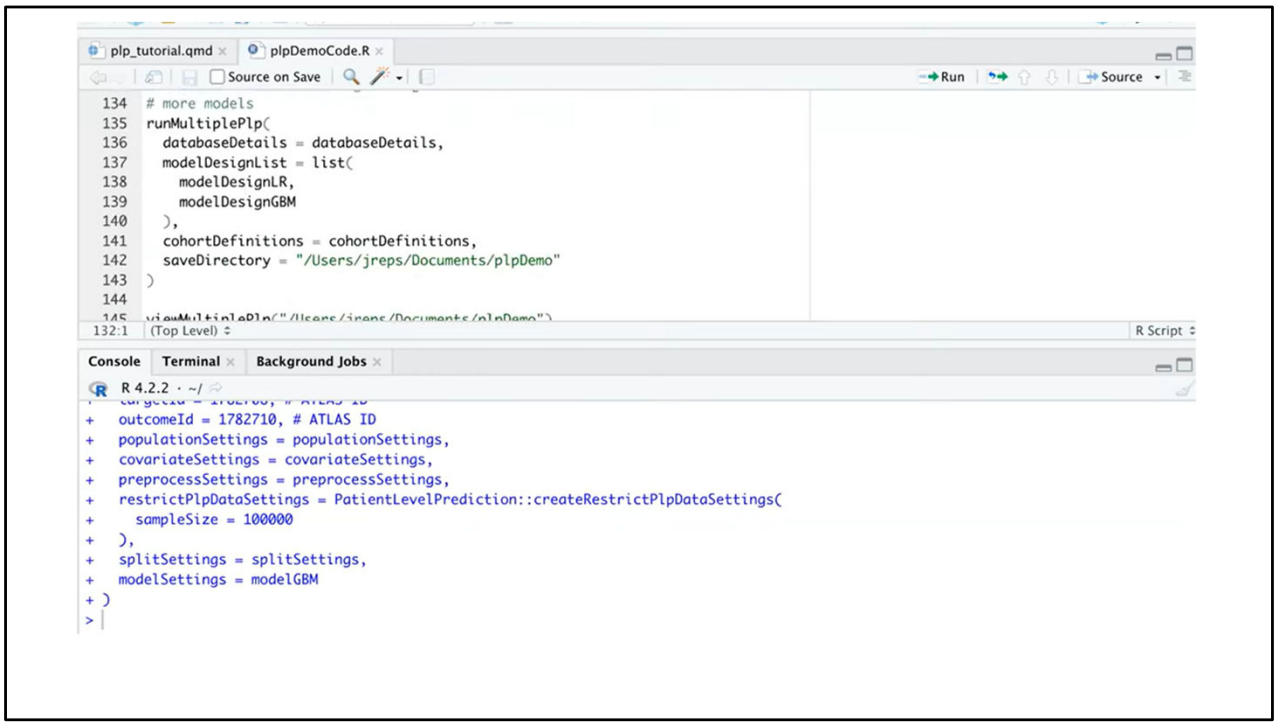

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
45 # Once cohorts are generated we can run our
46 # model design using PatientLevelPrediction
47 library(PatientLevelPrediction)
48
49 databaseDetails <- createDatabaseDetails(
50 connectionDetails = connectionDetails,
51 cdmDatabaseSchema = cdmDatabaseSchema,
52 cdmDatabaseId = 'mdcd',
53 cohortDatabaseSchema = cohortDatabaseSchema,
54 cohortTable = cohortTable,
55 outcomeDatabaseSchema = cohortDatabaseSchema,
49:1
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
$cohortSummaryStatsTable
[1] "plp_demo_table_summary_stats"
$cohortCensorStatsTable
[1] "plp_demo_table_censor_stats"
> # Once cohorts are generated we can run our
> # model design using PatientLevelPrediction
> library(PatientLevelPrediction)
>

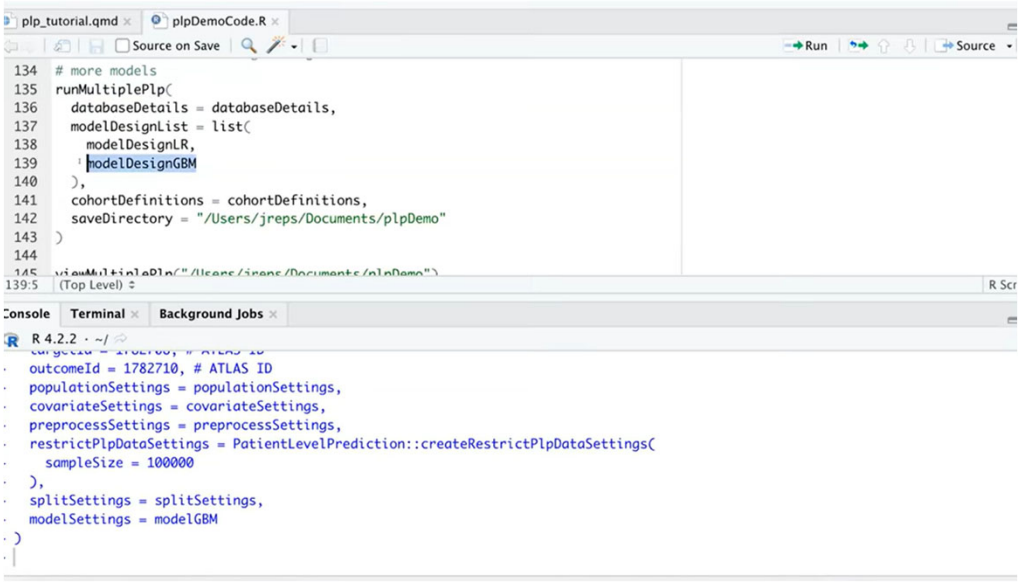

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
47 library(PatientLevelPrediction)
48
49 databaseDetails <- createDatabaseDetails(
50 connectionDetails = connectionDetails,
51 cdmDatabaseSchema = cdmDatabaseSchema,
52 cdmDatabaseId = 'mdcd',
53 cohortDatabaseSchema = cohortDatabaseSchema,
54 cohortTable = cohortTable,
55 outcomeDatabaseSchema = cohortDatabaseSchema,
56 outcomeTable = cohortTable
57)
58
53:1 (Top Level)
Console
Terminal
Background Jobs
R 4.2.2 · ~/
$cohortSummaryStatsTable
[1] "plp_demo_table_summary_stats"
$cohortCensorStatsTable
[1] "plp_demo_table_censor_stats"
> # Once cohorts are generated we can run our
> # model design using PatientLevelPrediction
> library(PatientLevelPrediction)

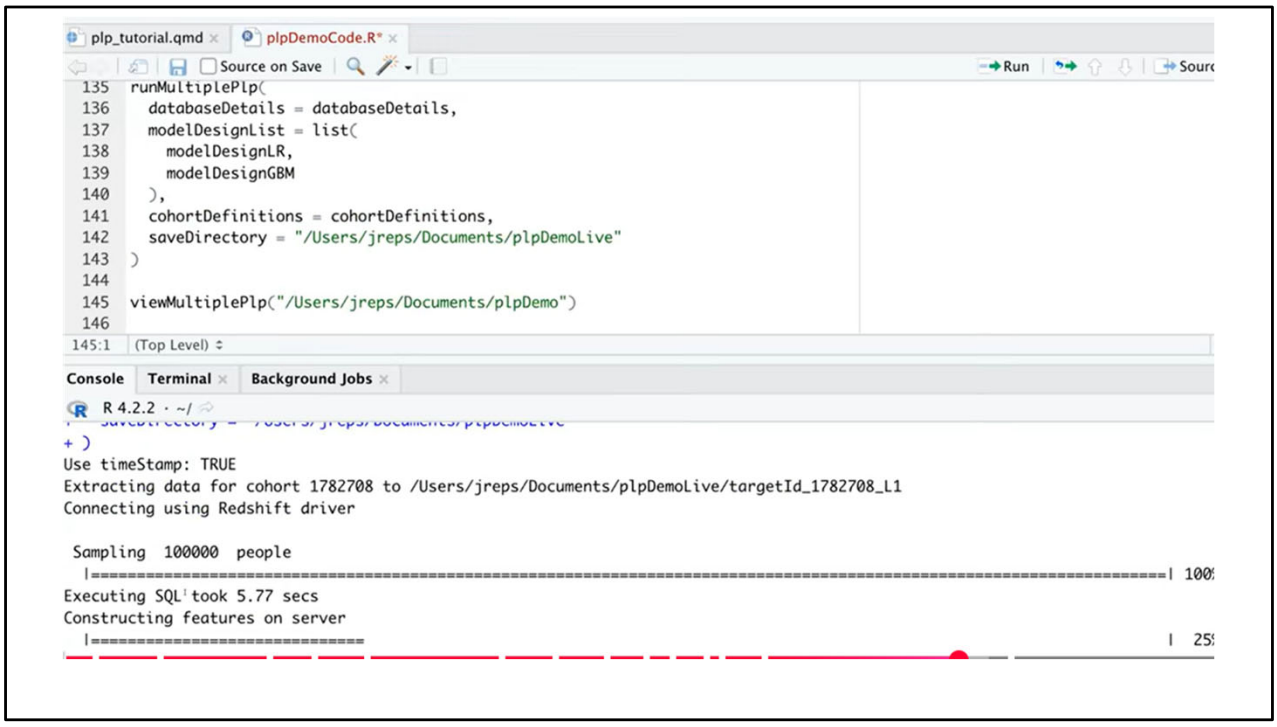

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
featureEngineering <- createFeatureEngineeringSettings(
type = 'none'
)
sampleSettings <- createSampleSettings(
type = 'none'
)
preprocessSettings <- createPreprocessSettings(
minFraction = 1/10000,
normalize = T,
removeRedundancy = T
93:1
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
+ splitSeed = 1535,
+ nfold = 3,
+ type = 'stratified' #subject/time
+)
> featureEngineering <- createFeatureEngineeringSettings(
+ type = 'none'
+)
> sampleSettings <- createSampleSettings(
+ type = 'none'
+)

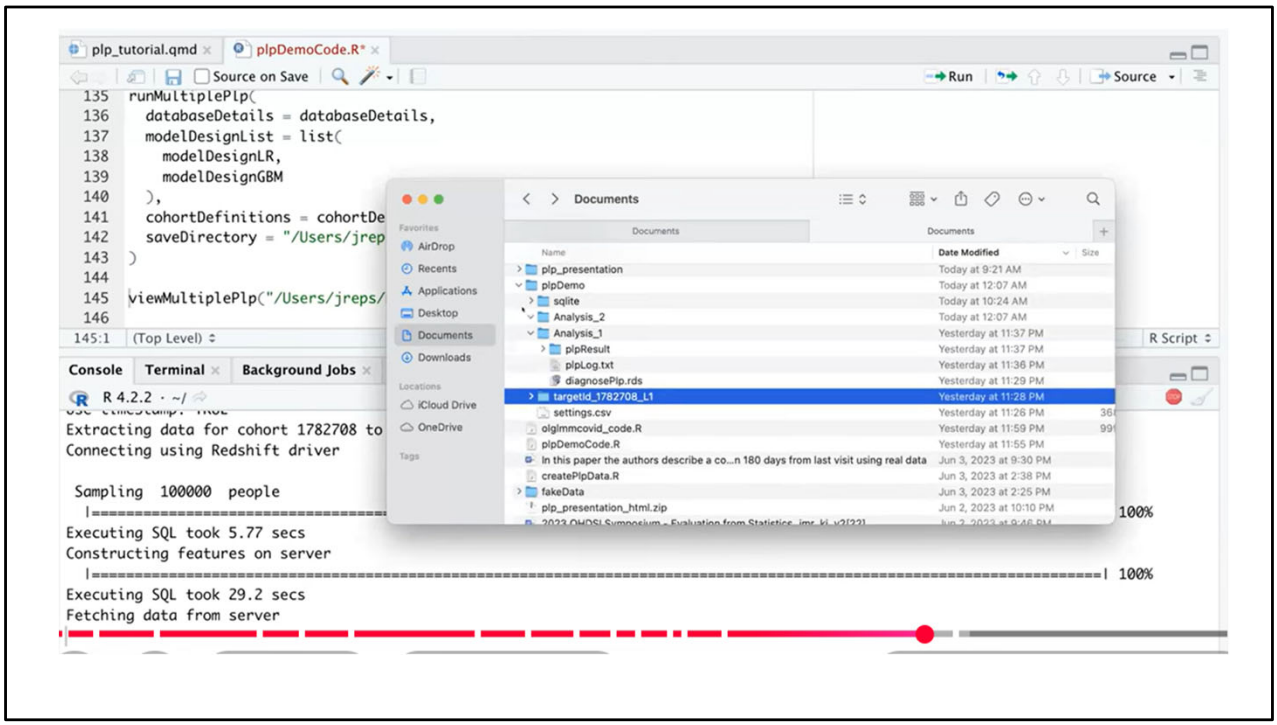

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
105
106 modelDesignLR <- createModelDesign(
107 targetId = 1782708, # ATLAS ID
108 outcomeId = 1782710, # ATLAS ID
109 populationSettings = populationSettings,
110 covariateSettings = covariateSettings,
111 preprocessSettings = preprocessSettings,
112 restrictPlpDataSettings = PatientLevelPrediction::createRestrictPlpDataSettings(
113 sampleSize = 100000
114),
115 splitSettings = splitSettings,
106:36 (Top Level)
Console
Terminal
Background Jobs
R 4.2.2 · ~/
> preprocessSettings <- createPreprocessSettings(
+ minFraction = 1/10000,
+ normalize = T,
+ removeRedundancy = T
+)
> modelLR <- setLassoLogisticRegression()
> # Option 2: Gradient Boosting Machines
> modelGBM <- setGradientBoostingMachine(
+ maxDepth = c(2,4,10,17)
+)

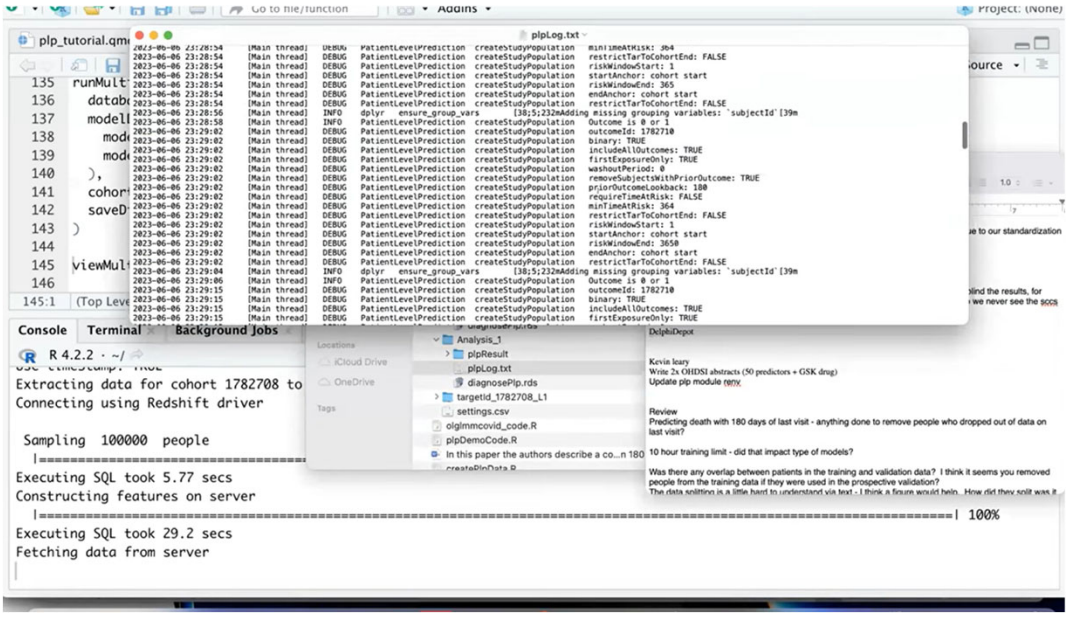

Go to file/function
Addins
Project: (N
plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
120 targetId = 1782708, # ATLAS ID
121 outcomeId = 1782710, # ATLAS ID
122 populationSettings = populationSettings,
123 covariateSettings = covariateSettings,
124 preprocessSettings = preprocessSettings,
125 restrictPlpDataSettings = PatientLevelPrediction::createRestrictPlpDataSettings(
126 sampleSize = 100000
127),
128 splitSettings = splitSettings,
129 modelSettings = modelGBM
130)
131
129:19 (Top Level)
R Scrip
Console
Terminal
Background Jobs
R 4.2.2 · ~/
+ outcomeId = 1782710, # ATLAS ID
+ populationSettings = populationSettings,
+ covariateSettings = covariateSettings,
+ preprocessSettings = preprocessSettings,
+ restrictPlpDataSettings = PatientLevelPrediction::createRestrictPlpDataSettings(
+ sampleSize = 100000
+),
+ splitSettings = splitSettings,
+ modelSettings = modelLR
+)
>

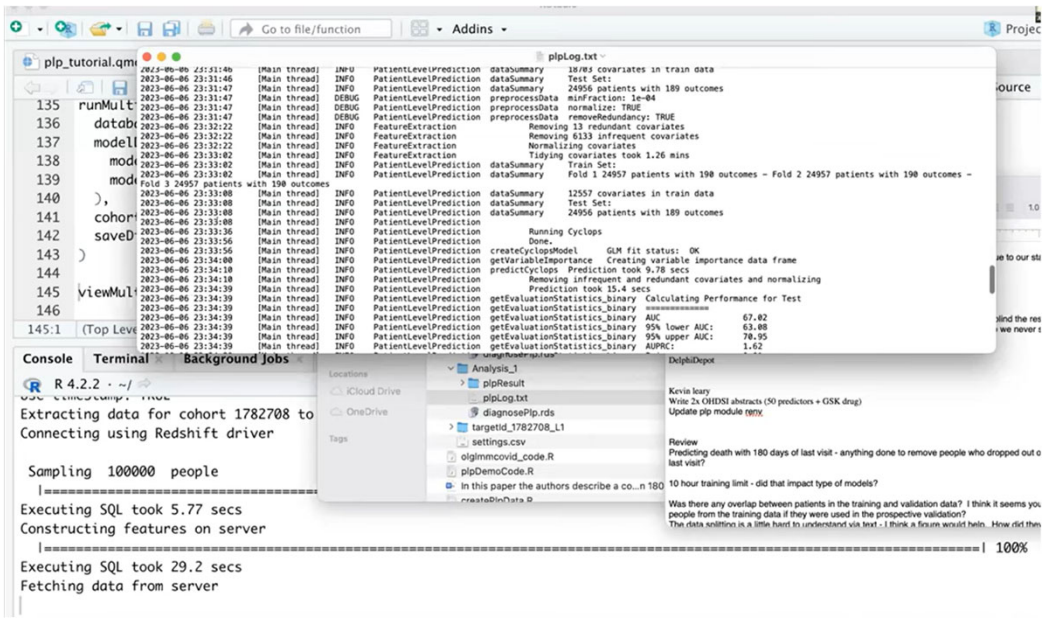

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
more models
runMultiplePlp(
databaseDetails = databaseDetails,
modelDesignList = list(
modelDesignLR,
modelDesignGBM
),
cohortDefinitions = cohortDefinitions,
saveDirectory = "/Users/jreps/Documents/plpDemo"
)
132:1
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
+ outcomeId = 1782710, # ATLAS ID
+ populationSettings = populationSettings,
+ covariateSettings = covariateSettings,
+ preprocessSettings = preprocessSettings,
+ restrictPlpDataSettings = PatientLevelPrediction::createRestrictPlpDataSettings(
+ sampleSize = 100000
+),
+ splitSettings = splitSettings,
+ modelSettings = modelGBM
+)
>

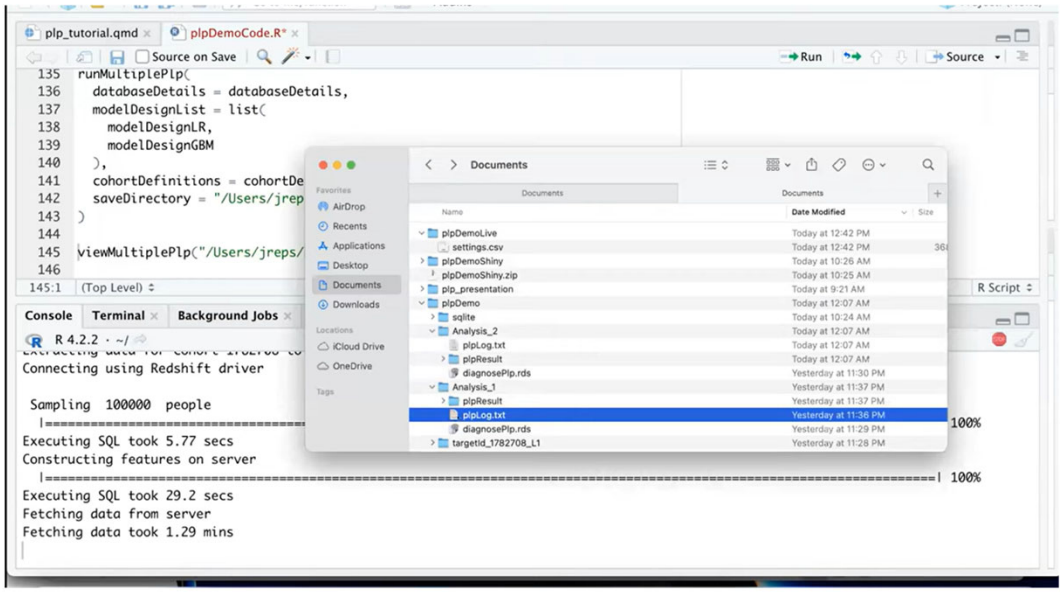

plp_tutorial.qmd
plpDemoCode.R
Source on Save
Run
Source
134 # more models
135 runMultiplePlp(
136 databaseDetails = databaseDetails,
137 modelDesignList = list(
138 modelDesignLR,
139 modelDesignGBM
140),
141 cohortDefinitions = cohortDefinitions,
142 saveDirectory = "/Users/jreps/Documents/plpDemo"
143)
144
139:5 (Top Level)
Console
Terminal
Background Jobs
R 4.2.2 · ~/
outcomeId = 1782710, # ATLAS ID
populationSettings = populationSettings,
covariateSettings = covariateSettings,
preprocessSettings = preprocessSettings,
restrictPlpDataSettings = PatientLevelPrediction::createRestrictPlpDataSettings(
sampleSize = 100000
),
splitSettings = splitSettings,
modelSettings = modelGBM
)

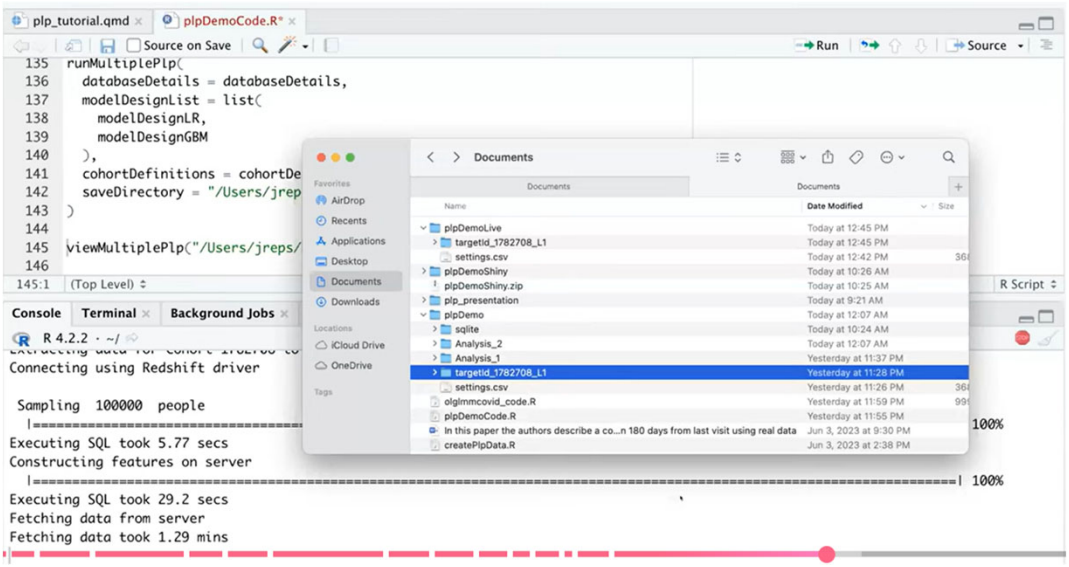

plp_tutorial.qmd
plpDemoCode.R*
Source on Save
Run
runMultiplePlp(
databaseDetails = databaseDetails,
modelDesignList = list(
modelDesignLR,
modelDesignGBM
),
cohortDefinitions = cohortDefinitions,
saveDirectory = "/Users/jreps/Documents/plpDemoLive"
)
viewMultiplePlp("/Users/jreps/Documents/plpDemo")
145:1
(Top Level)
Console
Terminal
Background Jobs
R 4.2.2 · ~/
+)
Use timeStamp: TRUE
Extracting data for cohort 1782708 to /Users/jreps/Documents/plpDemoLive/targetId_1782708_L1
Connecting using Redshift driver
Sampling 100000 people
Executing SQL took 5.77 secs
Constructing features on server


plp_tutorial.qmd
plpDemoCode.R*
Source on Save
Run
Source
runMultiplePlp(
databaseDetails = databaseDetails,
modelDesignList = list(
modelDesignLR,
modelDesignGBM
),
cohortDefinitions = cohortDe
saveDirectory = "/Users/jrep
)
viewMultiplePlp("/Users/jreps/
145:1
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
Extracting data for cohort 1782708 to
Connecting using Redshift driver
Sampling 100000 people
100%
Executing SQL took 5.77 secs
Constructing features on server
100%
Executing SQL took 29.2 secs
Fetching data from server
Documents
Favorites
AirDrop
Recents
Applications
Desktop
Documents
Downloads
Locations
iCloud Drive
OneDrive
Tags
Name
Date Modified
Size
plp_presentation
Today at 9:21 AM
plpDemo
Today at 12:07 AM
sqlite
Today at 10:24 AM
Analysis_2
Today at 12:07 AM
Analysis_1
Yesterday at 11:37 PM
plpResult
Yesterday at 11:37 PM
plpLog.txt
Yesterday at 11:36 PM
diagnosePlp.rds
Yesterday at 11:29 PM
targetId_1782708_L1
Yesterday at 11:28 PM
settings.csv
Yesterday at 11:26 PM
olgImmcovid_code.R
Yesterday at 11:59 PM
plpDemoCode.R
Yesterday at 11:55 PM
In this paper the authors describe a co...n 180 days from last visit using real data
Jun 3, 2023 at 9:30 PM
createPlpData.R
Jun 3, 2023 at 2:38 PM
fakeData
Jun 3, 2023 at 2:25 PM
plp_presentation_html.zip
Jun 2, 2023 at 10:10 PM


plp_tutorial.qm
plpLog.txt
Console
Terminal
Background Jobs
R 4.2.2 · ~/
Extracting data for cohort 1782708 to
Connecting using Redshift driver
Sampling 100000 people
Executing SQL took 5.77 secs
Constructing features on server
100%
Executing SQL took 29.2 secs
Fetching data from server
Analysis_1
plpResult
plpLog.txt
diagnosePlp.rds
targetid_1782708_L1
settings.csv
olgimmcovid_code.R
plpDemoCode.R
Kevin leary
Write 2x OHDSI abstracts (50 predictors + GSK drug)
Update plp module renv
Review
Predicting death with 180 days of last visit - anything done to remove people who dropped out of data on last visit?
10 hour training limit - did that impact type of models?
Was there any overlap between patients in the training and validation data? I think it seems you removed people from the training data if they were used in the prospective validation?


plp_tutorial.qmd
plpLog.txt
Console
Terminal
Background Jobs
R 4.2.2 · ~/
Extracting data for cohort 1782708 to
Connecting using Redshift driver
Sampling 100000 people
Executing SQL took 5.77 secs
Constructing features on server
100%
Executing SQL took 29.2 secs
Fetching data from server
Analysis_1
plpResult
plpLog.txt
diagnosePlp.rds
targetId_1782708_L1
settings.csv
olgImmcovid_code.R
plpDemoCode.R
DelphiDepot
Kevin leary
Write 2x OHDSI abstracts (50 predictors + GSK drug)
Update plp module renv
Review
Predicting death with 180 days of last visit - anything done to remove people who dropped out o
last visit?
10 hour training limit - did that impact type of models?
Was there any overlap between patients in the training and validation data? I think it seems you
people from the training data if they were used in the prospective validation?


plp_tutorial.qmd
plpDemoCode.R*
Source on Save
Run
Source
runMultiplePlp(
databaseDetails = databaseDetails,
modelDesignList = list(
modelDesignLR,
modelDesignGBM
),
cohortDefinitions = cohortDe
saveDirectory = "/Users/jrep
)
viewMultiplePlp("/Users/jreps/
145:1
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
Connecting using Redshift driver
Sampling 100000 people
100%
Executing SQL took 5.77 secs
Constructing features on server
100%
Executing SQL took 29.2 secs
Fetching data from server
Fetching data took 1.29 mins
Documents
Favorites
AirDrop
Recents
Applications
Desktop
Documents
Downloads
Locations
iCloud Drive
OneDrive
Tags
Name
Date Modified
Size
plpDemoLive
Today at 12:42 PM
settings.csv
Today at 12:42 PM
plpDemoShiny
Today at 10:26 AM
plpDemoShiny.zip
Today at 10:25 AM
plp_presentation
Today at 9:21 AM
plpDemo
Today at 12:07 AM
sqlite
Today at 10:24 AM
Analysis_2
Today at 12:07 AM
plpLog.txt
Today at 12:07 AM
plpResult
Today at 12:07 AM
diagnosePlp.rds
Yesterday at 11:30 PM
Analysis_1
Yesterday at 11:37 PM
plpResult
Yesterday at 11:37 PM
plpLog.txt
Yesterday at 11:36 PM
diagnosePlp.rds
Yesterday at 11:29 PM
targetid_1782708_L1
Yesterday at 11:28 PM


plp_tutorial.qmd
plpDemoCode.R*
Source on Save
Run
Source
runMultiplePlp(
databaseDetails = databaseDetails,
modelDesignList = list(
modelDesignLR,
modelDesignGBM
),
cohortDefinitions = cohortDe
saveDirectory = "/Users/jrep
)
viewMultiplePlp("/Users/jreps/
145:1
(Top Level)
R Script
Console
Terminal
Background Jobs
R 4.2.2 · ~/
Connecting using Redshift driver
Sampling 100000 people
100%
Executing SQL took 5.77 secs
Constructing features on server
100%
Executing SQL took 29.2 secs
Fetching data from server
Fetching data took 1.29 mins
Documents
Favorites
AirDrop
Recents
Applications
Desktop
Documents
Downloads
Locations
iCloud Drive
OneDrive
Tags
Name
Date Modified
Size
plpDemoLive
targetid_1782708_L1
settings.csv
plpDemoShiny
plpDemoShiny.zip
plp_presentation
plpDemo
sqlite
Analysis_2
Analysis_1
targetid_1782708_L1
settings.csv
olgimmcovid_code.R
plpDemoCode.R
In this paper the authors describe a co...n 180 days from last visit using real data
createPlpData.R
Today at 12:45 PM
Today at 12:45 PM
Today at 12:42 PM
Today at 10:26 AM
Today at 10:25 AM
Today at 9:21 AM
Today at 12:07 AM
Today at 10:24 AM
Today at 12:07 AM
Yesterday at 11:37 PM
Yesterday at 11:28 PM
Yesterday at 11:26 PM
Yesterday at 11:59 PM
Yesterday at 11:55 PM
Jun 3, 2023 at 9:30 PM
Jun 3, 2023 at 2:38 PM

Using the OHDSI Analysis Viewer for PLP

- Exploring & communicating PatientLevelPrediction results
- Interactive Shiny app for PLP results
- Part of the OHDSI ecosystem
- Focused on model evaluation & interpretation

Where the Analysis Viewer Fits

- ATLAS → cohort & population definition
- PLP (R package) → model training & validation
- Analysis Viewer → result exploration & reporting

What You Need Before Launching

Completed PLP run

savePlpResult = TRUE

- • Results directory with performance, covariates, model

Launching the Viewer

- library(AnalysisViewer)
- viewPlp(resultDirectory = "path/to/results")

Key Tabs & How to Use Them

Overview: sanity check

Performance: AUC, PR, ROC

Calibration: slope & intercept

Covariates: face validity

Predictions: risk distributions

Common Pitfalls & Best Practices

- • Empty Viewer → wrong directory
- • Over-optimistic AUC → check external validation
- • Poor calibration → consider recalibration
- • Use Viewer for QA, not just presentation